# Neural Network Verification

## A Guide for Researchers and Practitioners

**ThanhVu (Vu) Nguyen** and **Hai Duong**

George Mason University

# Contents and Summary

# Preface

# 1 Neural Networks

## 1.1 Basics of Neural Networks

A *neural network* (**NN**) [1] consists of an input layer, multiple hidden layers, and an output layer. Each layer has a number of neurons, each connected to neurons in the next layer through a predefined set of weights (computed during training). A *Deep Neural Network* (**DNN**) is an NN with *two* or more hidden layers.

The output of an NN is obtained by iteratively computing the values of neurons in each layer. The value of a neuron in the input layer is the input data. The value of a neuron in the hidden layers is computed by applying an *affine transformation* (Sec. 1.3) to values of neurons in the previous layers, then followed by an *activation function* (Sec. 1.4) such as ReLU and Sigmoid. The value of a neuron in the output layer is computed similarly but may skip the activation function.

## 1.2 NN as a Function

We can view an NN as a function that maps input vectors to output vectors:

$$f : \mathbb{R}^n \to \mathbb{R}^m \tag{1}$$

where $n$ is the number of input neurons and $m$ is the number of output neurons. The neurons in the input layer are the inputs to the function, and the neurons in the output layer are the outputs of the function. The neurons in the hidden layers are also functions that transform the inputs from the previous layer to produce outputs for the next layer.

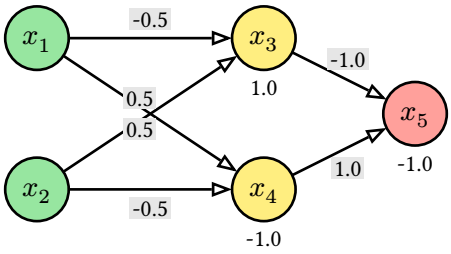



Fig. 1: A simple network with two inputs $x_1, x_2$, two hidden neurons $x_3, x_4$, and one output neuron $x_5$.

**Example 1.2.1**

Fig. 1 shows an NN with two inputs $x_1, x_2 \in \mathbb{R}$, two hidden neurons $x_3, x_4$ in one hidden layer, and one output neuron $x_5$. The connections between the neurons are weighted edges, and the biases are shown below each neuron.

- The hidden neurons compute:

$$x_3 = \text{relu}(-0.5x_1 + 0.5x_2 + 1.0), x_4 = \text{relu}(0.5x_1 - 0.5x_2 - 1.0), \tag{2}$$

  where $\text{relu}(x) = \max(x, 0)$ is the ReLU activation function.
- The output neuron computes:

$$x_5 = -1.0 \cdot x_3 + 1.0 \cdot x_4 - 1.0. \tag{3}$$

Thus, this NN computes a function $f : \mathbb{R}^2 \to \mathbb{R}$ where:

$$f(x_1, x_2) = -\text{ relu}(-0.5x_1 + 0.5x_2 + 1.0) + \text{relu}(0.5x_1 - 0.5x_2 - 1.0) - 1.0. \tag{4}$$

◊

## 1.3 Affine Transformation

The affine transformation (AF) of a neuron consists of a *linear combination*—a weighted sum $\sum$—of its inputs, followed by the addition of a bias term. More specifically, for a neuron with weights $w_1, ..., w_n$, bias $b$, and inputs $v_1, ..., v_n$ from the previous layer, the AF computes:

$$f(v_1, v_2, ..., v_n) = \sum_{i=1}^{n} w_i v_i + b. \tag{5}$$

**Example 1.3.1**

In Fig. 1, neuron $x_3$ receives inputs $x_1$ and $x_2$ with weights $-0.5$, $0.5$, and bias $1.0$, so its AF is $x_3 = -0.5x_1 + 0.5x_2 + 1.0$.

◊

## 1.4 Activation Functions

Activation functions in NNs are mathematical functions applied to the output of a neuron after the affine transformation. They introduce non-linearity to the network, allowing it to learn complex patterns in the data. Without activation functions, an NN would simply be a linear model, regardless of the number of layers, and would not be able to capture intricate relationships in the data.

Tab. 1 summarizes the most common activation functions used in NNs, their equations, output ranges, and key uses.

| Name | Equation | Output Range | Key Use |
|---|---|---|---|
| ReLU | $\max(0, x)$ | $[0, \infty)$ | Hidden layers, fast train |
| Sigmoid | $\frac{1}{1+e^{-x}}$ | $(0, 1)$ | Binary classification |
| Tanh | $\tanh(x)$ | $(-1, 1)$ | Hidden layers, zero-centered |
| Softmax | $\frac{e^{x_i}}{\sum_j e^{x_j}}$ | $(0, 1), \sum_i = 1$ | Multi-class output |

Tab. 1: Summary of Common Neural Network Activation Functions

### 1.4.1 ReLU (Rectified Linear Unit)

ReLU is a widely used activation function in neural networks. It is defined as:

$$\text{relu}(x) = \max(0, x) = \begin{cases} 0 \text{ if } x \leq 0 \\ x \text{ if } x > 0 \end{cases} \tag{6}$$

ReLU(x) is called piecewise linear because it consists of two linear segments as shown in Fig. 3: (i) a constant function that always return 0 when $x \leq 0$, (ii) an identity function that returns $x$ when $x > 0$. A ReLU activated neuron is said to be *active* if its input is greater than zero and *inactive* otherwise.

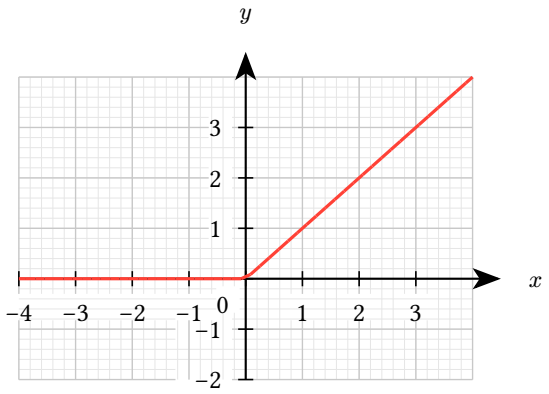



Fig. 3: ReLU (Rectified Linear Unit) function.

***Logical encoding*** ReLU can be encoded using the following logical formula:

$$y = \text{relu}(x) \iff (x \leq 0 \land y = 0) \lor (x > 0 \land y = x) \tag{7}$$

In other words, if $x \leq 0$, $y$ must be 0; otherwise, $y$ must equal $x$.

**Example 1.4.1.1**

In Z3, we can declare ReLU using `If()`

```
import z3
def relu(x): return z3.If(x <= 0, 0, x)

relu(-1.2)  # returns 0
relu(0)     # returns 0
relu(2.8)   # returns 2.8
```

◊

***Nonlinear Property*** Despite being piecewise linear, ReLU is **nonlinear** because it does not satisfy the two core properties of a linear function:

- *Additivity:* $\text{relu}(x + y) \neq \text{relu}(x) + \text{relu}(y)$ in general.
- *Homogeneity:* $\text{relu}(\alpha x) \neq \alpha \cdot \text{relu}(x)$ when $\alpha < 0$.

In simpler terms, ReLU is nonlinear because it does not form a straight line. It has a **kink**—a sharp bend—when $x = 0$, where the slope changes abruptly from $0$ to $1$. This discontinuity in the derivative prevents the function from being globally linear.

This non-linearity makes NNV difficult. In fact, verifying networks with ReLU is NP-complete, as shown in Sec. 3.2. We will use ReLU throughout this book as the default activation function for hidden neurons in an NN.

### 1.4.2 Sigmoid

Sigmoid is a smooth—i.e., continuous and differentiable—non-linear activation function that maps any real value to the range $(0, 1)$. It is continuous, meaning that small changes in the input will result in small changes in the output, and differentiable, meaning that it has a well-defined derivative at every point. Sigmoid is often used in the output layer of a binary classification problem.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \tag{8}$$

**Example 1.4.2.1**

$\text{sigmoid}(-1.2) \approx 0.23$, $\text{sigmoid}(0) = 0.5$, and $\text{sigmoid}(2.8) \approx 0.94$. This means that the sigmoid function maps $-1.2$ to a value close to $0$, $0$ to $0.5$, and $2.8$ to a value close to $1$. ◊

### 1.4.3 Softmax

**Example 1.4.3.1**

Use Z3 to encode the network in Fig. 1 and compute its output $x_5$ for the input $(x_1, x_2) = (2.0, 0.5)$ and find an input that maximizes the output $x_5$.

```
from z3 import Real, If, Solver, Optimize, sat

x1, x2, = Real('x1'), Real('x2')
x3, x3_, = Real('x3'), Real('x3_')
x4, x4_ = Real('x4'), Real('x4_')
x5 = Real('x5')

l2a = x3 = -0.5 * x1 + 0.5 * x2 + 1.0
l2a_ = x3_ = If(x3 > 0, x3, 0)
l2b = x4 =  0.5 * x1 - 0.5 * x2 - 1.0
l2b_ = x4_ = If(x4 > 0, x4, 0)
l3 = x5 = -1.0 * x3_ + 1.0 * x4_ - 1.0

# output for input (x1, x2) = (2.0, 0.5)
s = Solver()
s.add(l2a); s.add(l2b)
s.add(l2a_); s.add(l2b_); s.add(l3)

# specify input values
s.add(x1 = 2.0); s.add(x2 = 0.5)
if s.check() = sat:
  print(s.model())

# find an input that maximizes the output x5
o = Optimize()
o.add(l2a); o.add(l2b)
o.add(l2a_); o.add(l2b_); o.add(l3)

# maximize x5
o.maximize(x5)
if o.check() = sat:
  print(o.model())
```

◊

**Exercise 1.4.3.2**

Use Example 1.4.3.1 as an example and try the following exercises:

1. Use Z3 to encode the network in Fig. 1 and compute the outputs of *all* neurons for the input $(x_1, x_2) = (-1.3, 3.5)$.
2. Use Z3 to find an input $(x_1, x_2)$ that *maximizes* the output $x_5$ of the network in Fig. 1.
3. Add another input $x_{\text{new}}$ and update the network accordingly. Then, use Z3 to find an input $(x_1, x_2, x_{\text{new}})$ that *minimizes* the output $x_5$ of the updated network.

♡

## 1.5 Neural Network Architectures and Layers

NNs vary in architecture depending on how information flows through them and how computations are structured. Most common models are variations of the *feedforward network*, with additional structures or constraints layered on top. Tab. 2 summarizes several common NN architectures and their typical application domains.

| Name | Acronym | Typical Applications |
|---|---|---|
| Feedforward NN | FNN / MLP | General function approximation, tabular data |
| Convolutional NN | CNN | Image processing, video analysis |
| Residual NN | ResNet | Deep image recognition, medical imaging |
| Recurrent NN | RNN | Sequence modeling, NLP, time series |
| Transformer | – | NLP, summarization, code generation, vision |
| Graph NN | GNN | Graph-structured data, molecule modeling, recommendation |

Tab. 2: Popular NN Architectures and Applications

### 1.5.1 Feedforward Neural Networks (FNNs)

In an FNN, information flows in one direction: from the input layer, through one or more hidden layers, and finally to the output layer. There are no loops or cycles in the computation graph.

Widely used feedforward architectures include fully connected, convolutional, and residual networks. Each architecture has its own strengths and is suited for different types of tasks.

***Fully Connected NNs (FCNs)*** In FCNs, each neuron in a layer is connected to every neuron in the next layer. Thus, every neuron in the input layer is connected to every neuron in the first hidden layer, every neuron in the first hidden layer is connected to every neuron in the second hidden layer, and so on, until the output layer. Fully connected NNs, sometimes called *dense networks*, are the most basic type of FNNs and are commonly used for tasks like classification.

**Example 1.5.1.1 (Classifier)**

A common use case for FCNs is classification. They take an input, e.g., pixels of an image, and predict what the image is (e.g., cat, dog, car). The output layer represents the probabilities of each class (e.g., $y_1$ is the probability of cat, $y_2$ is the probability of dog). Moreover, the outputs are often passed through a `softmax` activation function (Sec. 1.4.3) to convert them into probabilities that sum to 1, and the class with the highest probability is chosen as the predicted label.

◊

***Convolutional NNs (CNNs)*** CNNs replace fully connected layers with *convolutional layers*, which apply local filters across the input space. In CNNs, each neuron receives several inputs, takes a weighted sum over them, passes it through an activation function, and responds with an output. CNNs are commonly used in computer vision and image processing. Despite their local structure, CNNs remain feedforward: data flows forward without cycles.

### 1.5.2 Other NN Architectures

Not all NNs are feedforward. Some architectures introduce cycles, dynamic connections, or non-Euclidean[1] data structures (e.g., graphs).

#### 1.5.2.1 Recurrent Neural Networks (RNNs)

RNNs, often used in natural language processing (NLP) and speech recognition, are designed to recognize patterns in sequences of data. RNNs have *loops* in them, allowing information to be sent forward and backward.

Currently, most NNV work tackles RNNs by *unrolling* them into an equivalent feedforward network. While this allows us to apply feedforward verification techniques, it can lead to an exponential increase in the size of the network, making verification more challenging.

#### 1.5.2.2 Transformers

Transformers are designed for long-range dependencies using *self-attention* rather than recurrence. They dominate applications in natural language processing and are increasingly used in vision and reinforcement learning.

#### 1.5.2.3 Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs) operate on graphs (instead of vectors like FNNs or sequences like RNNs). It uses message passing between nodes in the graph and allows each node to aggregate information from its neighbors. GNNs are used in applications involving structured data like molecules or social networks. Currently, most NNV tools do not support GNNs, and we will not focus on them in this book. However, they are an important area of research in the broader field of NNV.

## 1.6 Representing Neural Networks with ONNX

**ONNX** (Open Neural Network Exchange) [2] is an open source and widely adopted standard for representing neural networks. It provides a common format for representing the structure and parameters of neural networks, enabling interoperability between different ML frameworks and tools.

ONNX operators that cover most sequential feedforward networks include:

- `Add, Sub, Gemm, MatMul`: basic arithmetic and matrix operations.
- `ReLU, Sigmoid, SoftMax`: activation functions.
- `AveragePool, MaxPool, Flatten, Reshape`: tensor manipulation.
- `Conv, BatchNormalization, LRN`: CNN layers and normalizations.
- `Concat, Dropout, Unsqueeze`: tensor operations.

[1] Euclidean data refers to data that can be represented in a flat, two-dimensional space, such as images.

**Example 1.6.1**

For the network in Fig. 1, the ONNX representation would include:

- Input: $x_1, x_2$.
- Hidden layer: $x_3 = \text{relu}(-0.5x_1 + 0.5x_2 + 1.0)$, $x_4 = \text{relu}(0.5x_1 - 0.5x_2 + 1.0)$.
- Output: $x_5 = -x_3 + x_4 - 1$.

The ONNX representation would look like:

```
ir_version: 9
opset_import { version: 13 }
graph example_nn {
  input: "x"
  output: "x5"

  node { op_type: "Gemm" input: "x" input: "W1" input: "b1" output: "h1" }   #
-0.5*x1+0.5*x2+1
  node { op_type:  "relu" input: "h1" output: "x3" }

  node { op_type: "Gemm" input: "x" input: "W2" input: "b2" output: "h2" }   #
0.5*x1-0.5*x2+1
  node { op_type:  "relu" input: "h2" output: "x4" }

  node { op_type: "Neg" input: "x3" output: "nx3" }
  node { op_type: "Add" input: "nx3" input: "x4" output: "s1" }
  node { op_type: "Add" input: "s1" input: "c_minus_1" output: "x5" }

  initializer { name: "W1" values: [-0.5, 0.5] }
  initializer { name: "b1" values: [1.0] }
  initializer { name: "W2" values: [0.5, -0.5] }
  initializer { name: "b2" values: [1.0] }
  initializer { name: "c_minus_1" values: [-1.0] }
}
```

◊

# 2 Properties

Similar to traditional software systems, neural networks (NNs) have desirable properties to ensure the network behaves as expected. These could be specific to the applications modeled by the network, e.g., safety properties for a network modelling a collision avoidance system, or general properties that are desired by all networks, e.g., robustness to small perturbations in the input data.

Below we will discuss properties that are relevant to the verification of NNs. Specifically, these properties can be expressed in a formal language supported by a NNV tool. Additional properties can be found in the literature cite{seshia2018formal}.

## 2.1 Definition

As described in Sec. 1, NNs define functions of the form $f : \mathbb{R}^n \to \mathbb{R}^m$, where $n$ is the dimension of the input and $m$ is the dimension of the output. Thus, the properties or specifications of an NN—similarly to properties of software program—are defined in terms of its input and output:

"*For any input $x \in \mathbb{R}^n$ satisfying a precondition $P$, the neural network should produce an output $f(x) \in \mathbb{R}^m$ that satisfies a postcondition $Q$.*"

This says that if the input $x$ satisfies the precondition $P$, then the output $f(x)$ should satisfy the postcondition $Q$.

## 2.2 Common Properties in Neural Networks

We now define some commonly studied properties in NNs verification.

### 2.2.1 Robustness

**Robustness** ensures that small changes in the input do not drastically change the output. This is a desirable property for all neural networks, especially classifiers, where we want to ensure that similar inputs yield similar outputs. For example, a slightly blurred image of a red light should still be classified as a red light.

There are two types of robustness properties: emph{local} (robustness around a chosen point) and emph{global} (robustness everywhere).

#### 2.2.1.1 Local Robustness

A neural network $f$ is $\varepsilon$-locally robust at point $x$ with respect to norm $\|\cdot\|$ if

$$\forall x', \quad \|x - x'\| \leq \varepsilon \Rightarrow f(x) = f(x'). \tag{9}$$

where $|x - x'| \leq \varepsilon$ indicates that the difference between the two points is within a certain (small) threshold $\varepsilon$.

Thus local robustness says that a network is robust if all nearby inputs $x'$ (within radius $\varepsilon$) are classified the same as $x$. In other words, no small perturbation around this specific point $x$ will fool the classifier.

Local robustness is what most adversarial robustness papers mean, e.g., checking whether an image of a cat is still classified as a cat under small pixel noise.

**Example 2.2.1.1 (Local Robustness: Image Classification)**

Consider a neural network $f$ that classifies images into different categories (e.g., dog, cat, etc.). A robustness property requires that if an input image $c$ is classified as a dog, then any perturbed image $x$ that is visually similar to $c$ should also be classified as a dog. This can be expressed as:

$$\forall x, \quad \|c - x\| \leq \varepsilon \Rightarrow f(x) = f(c) \tag{10}$$

◊

### 2.2.1.2 Global Robustness

A neural network $f$ is $\varepsilon$-globally robust with respect to norm $\|\cdot\|$ if

$$\forall x_1, x_2, \quad \|x_1 - x_2\| \leq \varepsilon \Rightarrow f(x_1) = f(x_2). \tag{11}$$

This says the property must hold for all pairs of inputs in the domain: whenever two points are within $\varepsilon$ of each other, they must share the same label.

Global robustness is a very strong definition, and in practice, almost impossible unless the network is trivial (e.g., outputs the same label everywhere).

**Problem 2.2.1.2 (Robustness: Image Classification)**

Suppose that a network classifies an input image $x$ as digit 7. We want to ensure that if the image is slightly perturbed (e.g., brightness changed by a small amount $\varepsilon$), the network still outputs 7.

*Solution.*

$$\forall x'.\ \|x - x'\| \leq \varepsilon \Rightarrow f(x') = f(x) = 7 \tag{12}$$

◊

### 2.2.2 Safety

**Safety** properties ensure that the network conforms to certain safety constraints. This is particularly important in safety-critical applications, such as autonomous vehicles or medical diagnosis systems, where incorrect outputs can lead to catastrophic consequences. %For example, in a self-driving car, we want to ensure that the car maintains a safe distance from other vehicles and pedestrians under all scenarios.

**Example 2.2.2.1 (Collision Avoidance System)**

A safety property in a collision avoidance system such as an autonomous vehicle might be that if the intruder is distant and significantly slower than us, then we stay below a certain velocity threshold. Formally, this can be expressed as:

$$d_{\text{intruder}} > d_{\text{threshold}} \land v_{\text{intruder}} < v_{\text{threshold}} \Rightarrow v_{\text{us}} < v_{\text{threshold}} \tag{13}$$

where $d_{\text{intruder}}$ is the distance to the intruder, $d_{\text{threshold}}$ is a predefined safe distance, $v_{\text{intruder}}$ is the speed of the intruder, and $v_{\text{us}}$ is our speed.

Unlike robustness properties, which are often desirable in all networks, safety properties are often *specific* to the application domain. For example, a safety property for an autonomous vehicle may not be relevant for a surgical robot.

◊

**Exercise 2.2.2.2 (Safety: Self-Driving Car)**

A network controlling a self-driving car on a highway might require that: If the car in front is at least 160 meters away and moving slower than us, then we should not accelerate.

♡

### 2.2.3 Other properties

#### 2.2.3.1 Consistency

**Consistency** requires that a NN behaves consistently when given semantically equivalent or related inputs.

**Example 2.2.3.1 (Logical Consistency in LLMs)**

Consider queries $q_1$, $q_2$, and $q_3$ about a person's age:

$q_1$ : How old is person X?

$q_2$ : What year was X born if they are currently Y years old?

$q_3$ : Will X be Z years old in 2025?

Thus, if the LLM outputs age $Y$ for $q_1$, then $q_2$ should output `current_year - Y`, and the answer to $q_3$ should be logically consistent with the stated age. This prevents scenarios where an LLM claims someone is 30 years old but was born in 1985 when the current year is 2024.

◊

#### 2.2.3.2 Monotonicity

**Monotonicity** ensures that the NN maintains a consistent ordering relationship between inputs and outputs: an increase in certain input features always leads to a non-decreasing output value. This property is important in applications where domain knowledge dictates logical ordering constraints, such as fairness-aware systems, medical diagnosis, and scientific applications where physical laws impose natural ordering relationships.

**Example 2.2.3.2 (Fairness)**

A network modelling the probability of admission to a university should be monotonically non-decreasing with respect to GPA and test scores, regardless of gender. Formally, for applicants with profiles $(p, s, g)$ and $(p', s', g')$ where $p, p'$ are GPAs, $s, s'$ are test scores, and $g, g'$ are gender indicators:

$$\begin{aligned} p \leq p' \land s = s' \land g \neq g' \Rightarrow f(p, s, g) \leq f(p', s', g') \\ s \leq s' \land p = p' \land g \neq g' \Rightarrow f(p, s, g) \leq f(p, s', g') \end{aligned} \tag{14}$$

where $f$ is the neural network computing admission probability. Additionally, for fairness:

$$f(p, s, \text{male}) = f(p, s, \text{female}) \qquad \forall p, s \tag{15}$$

ensuring that applicants with identical academic qualifications receive the same treatment regardless of gender.

◊

## 2.3 Counterexamples

A *counterexample* (**cex**) is a witness that falsifies the correctness property. Given the property defined in Sec. 2.1, a counterexample is an input $x$ that satisfies the precondition $P$ but produces an output $f(x)$ that violates the postcondition $Q$.

**Example 2.3.1 (Counterexample to Robustness Property)**

For local robustness property (Sec. 2.2.1):

$$f(x) \neq f(x') \land \|x - x'\| \leq \varepsilon \Rightarrow x' \quad \text{is a counterexample.} \tag{16}$$

The goal of NNV (Sec. 3) is to either prove that a property holds—no cex exist—or find a cex that violates the property.

◊

**Example 2.3.2 (Counterexample: Monotonicity in Admission))**

A network predicts admission probability to a university. Inputs: GPA $p$ and test score $s$.

We want: a higher GPA with same score should not decrease admission probability.

But we observe a case violating this:

- A: GPA = 3.0, score = 1500, prediction = 0.8
- B: GPA = 3.5, score = 1500, prediction = 0.6

Using the numbers in this violating case, write the monotonicity requirement and show how this is a counterexample.

Monotonicity requirement:

$$p \leq p' \wedge s = s' \Rightarrow f(p, s) \leq f(p', s') \tag{17}$$

Counterexample:

$$3.0 \leq 3.5 \wedge 1500 = 1500 \quad \text{but} \quad f(3.0, 1500) = 0.8 > f(3.5, 1500) = 0.6 \tag{18}$$

◊

## 2.4 The VNN-LIB Specification Language

The VNN-LIB specification [3], [4] defines a standard format to describe neural networks and their properties. This enables the sharing of benchmarks across different tools and platforms, facilitating evaluations and comparisons of their performance. VNN-COMPs [5] uses VNN-LIB to evaluate different neural network verification tools.

Specifically, VNN-LIB defines a common format for the following components:

1. **Neural Network (or model) representation** in the ONNX format [2].
2. **Property specification** in SMT-LIB format [6] (Sec. 1.6).

### 2.4.1 VNN-LIB: Property Specification Language

Verification tasks involve proving that the output of a network remains within some desired post-condition $\sigma$, given inputs within a bounded set $\Pi$.

***Formal Specification*** Let $\nu : D^{n_1 \times \ldots \times n_h} \to D^{m_1 \times \ldots \times m_k}$ be a neural network, and $x$ and $y$ its input and output tensors. A property is expressed as:

$$\forall x \in \Pi \to \nu(x) \in \sigma \tag{19}$$

This includes:

- **Precondition** $\Pi$: constraints on inputs.
- **Postcondition** $\sigma$: required properties of outputs.

Properties are encoded in **SMT-LIB2**, referencing input/output variable names consistent with ONNX.

# 3 The Neural Network Verification (NNV) Problem

**Definition 3.1 (NNV)**

Given a NN $N$ and a property $\varphi$, the NNV problem asks if $\varphi$ is a valid property[2] of $N$.

Typically, $\varphi$ is a formula of the form

$$\varphi_{\text{in}} \rightarrow \varphi_{\text{out}}, \tag{20}$$

where $\varphi_{\text{in}}$ is the *precondition*, i.e., a condition over the inputs of $N$, and $\varphi_{\text{out}}$ is the *postcondition*, i.e., a condition over the outputs of $N$.

An NNV tool attempts to find a *counterexample* input to $N$ that satisfies $\varphi_{\text{in}}$ but violates $\varphi_{\text{out}}$. If no such counterexample exists, $\varphi$ is a valid property of $N$. Otherwise, $\varphi$ is not valid and the counterexample can be used to retrain or debug the network [7].

♣

**Example 3.2 (Safety Property for Collision Avoidance System)**

In Example 2.2.2.1, we defined a safety property (Sec. 2.2.2) for a collision avoidance system:

$$d_{\text{intruder}} > d_{\text{threshold}} \wedge v_{\text{intruder}} < v_{\text{threshold}} \rightarrow v_{\text{us}} < v_{\text{threshold}}, \tag{21}$$

where $d_{\text{intruder}}$ is the distance to the intruder, $d_{\text{threshold}}$ is a predefined safe distance, $v_{\text{intruder}}$ is the speed of the intruder, and $v_{\text{us}}$ is our speed.

Here, the precondition is

$$\varphi_{\text{in}} = d_{\text{intruder}} > d_{\text{threshold}} \wedge v_{\text{intruder}} < v_{\text{threshold}}, \tag{22}$$

and the postcondition is

$$\varphi_{\text{out}} = v_{\text{us}} < v_{\text{threshold}}. \tag{23}$$

◊

[2] Sec. 2 provides various examples of properties.

**Example 3.3**

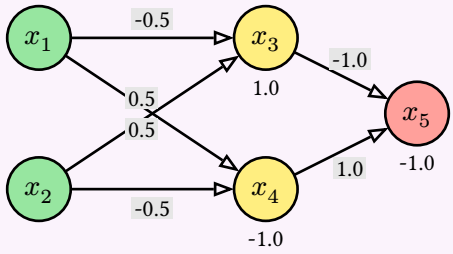



Fig. 4: A simple network (similar to Fig. 1).

For the network in Fig. 4, a *valid* property is that the output is $x_5 \leq 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$.

An *invalid* property is that $x_5 > 0$ for similar inputs. A counterexample showing this property violation is $\{x_1 = -1, x_2 = 2\}$, from which the network evaluates to $x_5 = -3.5$. ◊

## 3.1 Satisfiability Formulation and Checking

As with traditional software verification, NNV is often represented as a satisfiability problem, which can be solved using an SMT solver (e.g., Z3 [8]) or a MILP solver (e.g., Gurobi [9]).

***Formulation*** To formulate the problem, we first define a formula $\alpha$ to represent the network. Typically $\alpha$ is a conjunction of constraints representing the affine transformation (Sec. 1.3) and activation function (Sec. 1.4) of each neuron in the network. For example, for a fully-connected neural network (Sec. 1.5.1) with $L$ layers and $N$ ReLU neurons per layer, $\alpha$ is:

$$\alpha = \bigwedge (i \in [1, L], j \in [1, N]) v_{i,j} = \text{relu}\left( \sum_{k \in [1,N]} \left( w_{i-1,j,k} \cdot v_{i-1,k} \right) + b_{i,j} \right) \tag{24}$$

where $v_{i,j}$ is the output of the $j$-th neuron in layer $i$, $w_{i-1,j,k}$ is the weight connecting the $k$-th neuron in layer $i-1$ to the $j$-th neuron in layer $i$, and $b_{i,j}$ is the bias of the $j$-th neuron in layer $i$. The input layer is layer 0, i.e., $v_{0,j}$ are the input variables.

With this, the NNV problem (Definition 3.1) can be formulated as checking the validity of the following formula:

$$\alpha \Rightarrow (\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}) \tag{25}$$

Eq. 25 checks if network $N$ satisfies (implies) the property $\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$. This validity checking can be reduced to checking the satisfiability of the formula:

$$\alpha \wedge \varphi_{\text{in}} \wedge \neg \varphi_{\text{out}} \tag{26}$$

If Eq. 26 is unsatisfiable, then $\varphi$ is a valid property of $N$. Otherwise, $\varphi$ is not valid. Moreover, we can extract a counterexample for the original problem from the satisfying assignment of Eq. 26. This formulation is commonly used by NNV tools, as it allows us to leverage powerful SMT/MILP solvers to automatically check properties and find counterexamples.

**Problem 3.1.1**

Let $\alpha$ represent our network and the robustness property $|x - y| \leq \varepsilon \Rightarrow f(x) = f(y)$. Form the satisfiability formula (Eq. 26) that we need to check.

◊

**Problem 3.1.2**

Let $\alpha$ represent our network and the property $y > 0$ for any input $x_1 \in [r_1, r_2], x_2 \in [r_3, r_4]$. Form the satisfiability formula (Eq. 26) that we need to check.

◊

**Problem 3.1.3 (Validity Formulation)**

Show that the formula $\alpha \Rightarrow \varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$ in Eq. 25 is valid if and only if $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ (Eq. 26) is unsatisfiable.

First, do this by hand (on paper) by explicitly writing out the logical equivalences step by step. Then, verify your result using Z3, i.e., show that the negation of the first formula is equivalent to the second formula.

◊

**Example 3.1.4**

We represent the network in Fig. 4 as a formula $\alpha$:

$$\begin{aligned} x_3 &= \text{relu}(-0.5x_1 + 0.5x_2 + 1.0) \wedge \\ x_4 &= \text{relu}(0.5x_1 - 0.5x_2 - 1.0) \wedge \\ x_5 &= -x_3 + x_4 - 1.0 \end{aligned} \tag{27}$$

and the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$ as:

$$\begin{aligned} \varphi_{\text{in}} &= (-1 \leq x_1 \leq 1) \wedge (-2 \leq x_2 \leq 2) \\ \varphi_{\text{out}} &= (x_5 > 0) \end{aligned} \tag{28}$$

◊

***Satisfiability Solving*** We can check the satisfiability of $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ (Eq. 26) using constraint solving, e.g., the Z3 SMT solver. Sec. 4.1 shows how to perform symbolic execution and use an SMT solver to automatically generate this formula from a given network and property, and check its satisfiability. Sec. 4.2 shows how to encode the problem as MILP constraints, solvable using a MILP solver (e.g., Gurobi).

**Example 3.1.5**

For Example 3.1.4, the formula is satisfiable, i.e., the property is invalid, and the solver returns SAT. Any satisfying assignment, e.g., $x_1 = -1$ and $x_2 = 2$, is a counterexample to the property, as it satisfies the precondition $\varphi_{\text{in}}$ but violates the postcondition $\varphi_{\text{out}}$, i.e., $x_5 = -3.5$, which is $< 0$.

◊

**Problem 3.1.6**

Use Z3 to do Example 3.1.4. You might find Example 1.4.3.1 useful. Make sure that you also ask Z3 to find a counterexample violating the property (it does not have to be the same as above).

◊

**Problem 3.1.7**

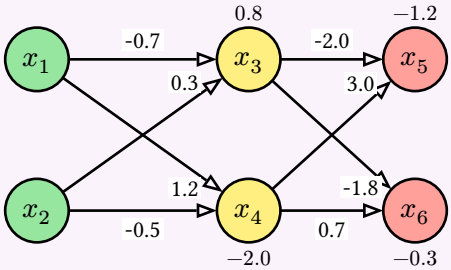



Fig. 6: A simple network with 2 inputs, 2 hidden ReLU neurons, and 2 outputs.

Consider the neural network in Fig. 6. Do the following:

1. Write the formula $\alpha$ representing the network.
2. Create the property $\varphi$ that the output $x_5 \geq x_6$ for any inputs $x_1 \in [-1, 1]$, $x_2 \in [-1, 1]$.
3. Generate the satisfiability formula $\alpha \wedge \overline{\varphi}$ as shown in Eq. 26.
4. Use Z3 to check the satisfiability of the formula. If it is satisfiable, extract a counterexample input that violates the property.

◊

## 3.2 Complexity

As shown in [10], [11] ReLU-based NNV is NP-complete by reducing the 3-SAT problem to it. This means that the problem of checking whether a given ReLU-based network satisfies a property is computationally hard, and no polynomial-time algorithm is known to solve it in the general case.

# 4 Constraint Solving

## 4.1 Symbolic Execution and SMT Solving

As described in Sec. 3.1 the Neural Network Verification (NNV) problem can be formulated as a satisfiability problem. Specifically, we encode the network as a logical formula, and use a constraint solver to check that formula satisfies the property of interest.

A straightforward and automated way to do this encoding is using *symbolic execution* (SE) [12], [13], a well-known software testing technique for finding bugs. SE executes a program on symbolic inputs, i.e., inputs represented as symbols rather than concrete values, and tracks the reachability of program state as symbolic expressions, i.e., logical formulae over symbolic inputs. The satisfiability of these formulae is then checked using an SMT solver, and satisfying assignments represent inputs leading to the undesirable (buggy) program state.

### 4.1.1 Symbolic Execution

We can adapt traditional SE to our problem by treating the network as a program and neurons as variables and executing the network on symbolic inputs. Affine transformations can easily be represented as logical formulae because they are linear functions. Activation functions such as ReLUs are translated to disjunctions of linear functions or if-then-else statements:

$$\begin{gathered} y = \text{relu}(x) \\ y = \max(x, 0) \\ y = \text{if } x > 0 \text{ then } x \text{ else } 0 \\ (x > 0 \wedge y = x) \vee (x \leq 0 \wedge y = 0) \end{gathered} \tag{29}$$

**Example 4.1.1.1**

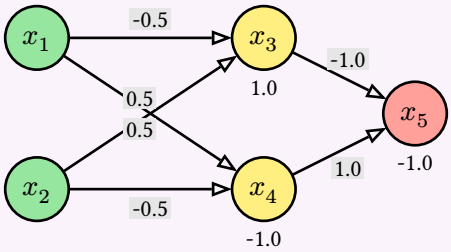



Fig. 8: A simple network with two inputs $x_1, x_2$, two hidden neurons $x_3, x_4$, and one output neuron $x_5$.

To create a logical formula representing the network in Fig. 8, we can symbolically execute the network on symbolic inputs $x_1, x_2$ and track the values of the neurons $x_3, x_4, x_5$ as a set (conjunction) of logical formulae. SE starts with the inputs $x_1$ and $x_2$ and computes the values of the neurons in the hidden layer, $x_3$ and $x_4$, using the affine transformations, followed by ReLUs. Finally, SE computes the output neuron $x_5$ as a linear combination of the hidden layer neurons.

$$\begin{aligned} x_5 &= -x_3 + x_4 - 1.0 \wedge \\ x_3 &= \max(-0.5x_1 + 0.5x_2 + 1.0, 0) \wedge \\ x_4 &= \max(0.5x_1 - 0.5x_2 - 1.0, 0) \end{aligned} \tag{30}$$

◊

### 4.1.2 SMT Solving

After obtaining the symbolic representation of the network, we can use an SMT solver [6] to check the satisfiability of the formula $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ as shown in Eq. 26, where $\alpha$ is the symbolic representation of the network, $\varphi_{\text{in}}$ is the precondition on the inputs, and $\varphi_{\text{out}}$ is the postcondition on the outputs.

The solver checks if there exists an assignment to the symbolic inputs that satisfies the formula. If such an assignment exists, it means that the property is violated, and we can extract a counterexample from the satisfying assignment. Otherwise, if no such assignment exists, the property is valid.

**Example 4.1.2.1**

To check that the network in Fig. 8 satisfies the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$ (Example 3.1.4), represented as:

$$\begin{gathered} \varphi_{\text{in}} = (x_1 \geq -1) \wedge (x_1 \leq 1) \wedge (x_2 \geq -2) \wedge (x_2 \leq 2) \\ \varphi_{\text{out}} = (x_5 > 0) \end{gathered} \tag{31}$$

We check the satisfiability of $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$:

$$\begin{aligned} & x_5 = -x_3 + x_4 - 1.0 \wedge \\ & x_3 = \max(-0.5x_1 + 0.5x_2 + 1.0, 0) \wedge \\ & x_4 = \max(0.5x_1 - 0.5x_2 - 1.0, 0) \wedge \\ & \quad (x_1 \geq -1) \wedge (x_1 \leq 1) \wedge (x_2 \geq -2) \wedge (x_2 \leq 2) \wedge (x_5 \leq 0) \end{aligned} \tag{32}$$

In this case, the SMT solver returns `sat` and a satisfying assignment, e.g., $x_1 = -1$ and $x_2 = 2$, which is a counterexample to the property. This means that for these inputs, the output $x_5 = -3.5$ violates the property $x_5 > 0$.

◊

### 4.1.3 Limitations

While using symbolic execution and SMT solving is a straightforward way to verify networks, it has several practical limitations:

- **Path Explosion and Scalability**: the number of paths that the solver has to analyze can grow exponentially with the number of ReLU-based neurons and layers as each ReLU, represented as a disjunction of linear functions, has two possible outputs for any input (i.e., the input value itself or 0). This leads to the notorious *path explosion* problem and becoming intractable for large networks. Programming assignment 1 (Sec. E.1) demonstrates this issue.
- **Non-linearity:** Other non-linear activation functions (Sec. 1.4), such as Sigmoid or Tanh, might not be easily representable as disjunctions of linear functions as ReLU. This can lead to complex formulae that are hard to reason about and/or result in a large search space for the SMT solver.
- **Precision Issues:** SMT solvers may struggle with precision when dealing with floating-point arithmetic, which is common in neural networks. This can lead to incorrect results or false positives/negatives in the verification process.

For these reasons, SMT solving is mostly used on toy examples, e.g., in a classroom setting. Modern NNV tools do not use SMT solving, and instead, rely on more efficient techniques such as abstraction (Sec. 5).

## 4.2 MILP

Instead of using SMT solving, we can encode the NNV problem as a Mixed Integer Linear Programming (MILP) constraints (Sec. D.1), and then invoke an MILP solver such as Gurobi [9] to check their satisfiability or *feasibility* (Sec. D.3.1).

### 4.2.1 ReLU Encoding

We first encode non-linear activation functions like ReLU using *binary indicator* variables and linear constraints. For each neuron, we introduce a binary variable (Sec. D.2.1) that indicates whether the neuron is "active" (output equals input) or "inactive" (output is zero). This transforms the non-linear `max(z,0)` operation into a set of linear inequalities controlled by the binary variable.

**Definition 4.2.1.1**

We define

- $z$: the pre-activation value, i.e., the value that goes into ReLU
- $\hat{z}$: the post-activation value, i.e., the output of ReLU
- $a \in \{0, 1\}$: a binary indicator variable encoding whether the neuron is active ($z \geq 0$) or inactive ($z < 0$)
- $l, u$: lower and upper bounds on $z$ ($l \leq z \leq u$).

♣

***Encoding*** The MILP encoding of $\hat{z} = \max(z, 0)$ is then (Example 4.2.1.3 shows a simpler example that does not involve bounds):

$$\begin{aligned} &\hat{z} \geq z \\ &\hat{z} \geq 0 \\ &\hat{z} \leq a \cdot u \\ &\hat{z} \leq z - l(1 - a) \\ &a \in \{0, 1\} \end{aligned} \tag{33}$$

These constraints enforce $\hat{z} = z$ when $a = 1$ (active) and $\hat{z} = 0$ when $a = 0$ (inactive), which captures the two possible ReLU outputs (0 or $z$). Note that constraints are linear and involve both continuous variable $\hat{z}$ and binary variable $a$.

### 4.2.2 Computing Concrete Bounds

The mentioned lower and upper bounds (Sec. 4.2.1)—*concrete* values $l$ and $u$ such that $l \leq z \leq u$ over the input $z$ to ReLU— are critical for ensuring that the binary indicator $a$ can only take on values that are *valid* given the possible range of $z$. For example, if $u < 0$, then $z$ is always negative, so the active phase ($a = 1$) is infeasible, and thus can be eliminated. Similarly, if $l \geq 0$, then $z$ is always active.

Either of these cases indicates the neuron is *stable*—always active or always inactive—and thus significantly simplifies the MILP problem because ReLU can be replaced with a linear function (identity or constant 0). Of course, if $l < 0 < u$, then both active and inactive phases are possible, and the MILP solver has to consider both cases. Sec. 5 discusses abstraction techniques to approximate ReLU outputs.

**Computing Bounds $l \leq e \leq u$**: Given a linear expression $e$

$$e = a_1 x_1 + a_2 x_2 + \ldots + b, \tag{34}$$

where each variable $x_i$ ranges over $[l_i, u_i]$, a simple way to compute the bounds $l \leq e \leq u$ is using *interval* arithmetic (Sec. 5.3):

- **For the lower bound** ($l_3$): For each $x_i$, use its upper bound $u_i$ if $a_i < 0$, and use its lower bound $l_i$ if $a_i \geq 0$.
- **For the upper bound** ($u_3$): For each $x_i$, use $l_i$ if $a_i < 0$, and use $u_i$ if $a_i \geq 0$.

This guarantees that the extreme values of $e$ are achieved at some corner of the input box.

**Example 4.2.2.1**

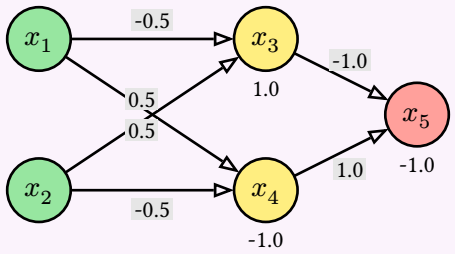



Fig. 10: A simple network (similar to Fig. 1.

Consider neuron $x_3$ in the network in Fig. 10. We have $x_3 = -0.5x_1 + 0.5x_2 + 1.0$ as the pre-activation value of $x_3$. The upper $u_3$ and lower $l_3$ bounds on $x_3$ over the input region $x_1 \in [-1, 1]$ and $x_2 \in [-2, 2]$ are:

$$\begin{aligned} z_3 &= -0.5x_1 + 0.5x_2 + 1.0 \\ l_3 &= -0.5(1) + 0.5(-2) + 1.0 = -0.5 \\ u_3 &= -0.5(-1) + 0.5(2) + 1.0 = 2.5 \end{aligned} \tag{35}$$

Notice that we use different pairs of values to compute the lower (1,-2) and upper (-1,2) bounds.

With the computed bounds, we encode the ReLU output of $x_3$:

$$\begin{aligned} &\hat{x}_3 \geq x_3 \\ &\hat{x}_3 \geq 0 \\ &\hat{x}_3 \leq a_3 \cdot u_3 \\ \Rightarrow\ &\hat{x}_3 \leq a_3 \cdot 2.5 \\ &\hat{x}_3 \leq x_3 - l_3(1 - a_3) \\ \Rightarrow\ &\hat{x}_3 \leq x_3 - (-0.5)(1 - a_3) \\ \Rightarrow\ &\hat{x}_3 \leq x_3 + 0.5(1 - a_3) \\ &a_3 \in \{0, 1\} \end{aligned} \tag{36}$$

Note that because $l_3 = -0.5$ and $u_3 = 2.5$, $a_3$ can be either 0 or 1, depending on the value of $x_3$. If we had different bounds, e.g., $l_3 = 0.1$, then $a_3$ would be 1, as $x_3$ can never be less than 0.1 (i.e., $x_3$ is a stable neuron because it is always active). Stable neurons replaces ReLU with either 0 or identity function, which significantly simplifies the problem.

### 4.2.3 Neural Network Encoding

We can encode a neural network as a set of MILP linear constraints as follows:

$$\begin{aligned}
&(a) \quad z^i = W^i \hat{z}^{(i-1)} + b^i; \\
&(b) \quad y = z^{\{L\}}; x = \hat{z}^{\{0\}}; \\
&(c) \quad \hat{z}^i_j \geq \{z\}^i_j; \quad \hat{z}^i_j \geq 0; \\
&(d) \quad a^i_j \in \{0, 1\}; \\
&(e) \quad \hat{z}^i_j \leq \{a\}^i_j u^i_j; \quad \hat{z}^i_j \leq z^i_j - l^i_j\big(1 - a^i_j\big);
\end{aligned} \tag{37}$$

where $x$ is input, $y$ is output, and $z^i$, $\hat{z}^i$, $W^i$, and $b^i$ are the pre-activation, post-activation, weight, and bias vectors for layer $i$ ($j$ is the neuron index in layer $i$).

Thus, we can encode a neural network as a set of linear constraints by:

1. Define the affine transformation computing the pre-activation value for a neuron in terms of outputs in the preceding layer;
2. Define the inputs and outputs in terms of the adjacent hidden layers;
3. Assert that post-activation values are non-negative and no less than pre-activation values;
4. Define that the neuron activation status indicator variables that are either 0 or 1; and
5. Define constraints on the upper, $u^{\{(i)\}}_j$, and lower, $l^{\{(i)\}}_j$, bounds of the pre-activation value of the $j$th neuron in the $i$th layer.

Deactivating a neuron, $a^i_j = 0$, simplifies the first of the (5) constraints to $\hat{z}^i_j \leq 0$, and activating a neuron simplifies the second to $\hat{z}^i_j \leq z^i_j$, which is consistent with the semantics of $\hat{z}^i_j = \max\big(z^i_j, 0\big)$.

**Example 4.2.3.1 (Full example)**

We use MILP to formulate and check if the network in Fig. 10 satisfies the property $x_5 > 0$ for any inputs $x_1 \in [-1, 1], x_2 \in [-2, 2]$. We do this by checking the feasibility of the MILP constraints encoding $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$, where $\alpha$ is the MILP encoding of the network, $\varphi_{\text{in}}$ is the precondition, and $\varphi_{\text{out}}$ is the postcondition. We do this in several steps:

1. **Encoding**: **precondition** $\varphi_{\text{in}}$ and **postcondition** $\neg\varphi_{\text{out}}$:

$$\varphi_{\text{in}} : -1 \leq x_1 \leq 1; \quad -2 \leq x_2 \leq 2, \quad \neg\varphi_{\text{out}} : x_5 \leq 0 \tag{38}$$

   **Hidden layer** (pre- and post-activation):

$$\begin{aligned} &z_3 = -0.5x_1 + 0.5x_2 + 1.0, \quad z_4 = 0.5x_1 - 0.5x_2 - 1.0 \\ &\hat{z}_3 \geq z_3, \quad \hat{z}_3 \geq 0, \quad \hat{z}_4 \geq z_4, \quad \hat{z}_4 \geq 0 \\ &a_3, a_4 \in \{0, 1\} \\ &\hat{z}_3 \leq a_3 \cdot u_3, \quad \hat{z}_3 \leq z_3 - l_3(1 - a_3), \quad \hat{z}_4 \leq a_4 \cdot u_4, \quad \hat{z}_4 \leq z_4 - l_4(1 - a_4) \end{aligned} \tag{39}$$

   **Output layer**: $x_5 = -\hat{z}_3 + \hat{z}_4 - 1.0$

2. **Computing** upper and lower bounds over input ranges $x_1 \in [-1, 1], x_2 \in [-2, 2]$:

$$\begin{aligned} z_3 &\in [-0.5 \cdot 1 + 0.5 \cdot (-2) + 1, -0.5 \cdot (-1) + 0.5 \cdot 2 + 1] = [-0.5, 2.5] \\ z_4 &\in [0.5 \cdot (-1) - 0.5 \cdot 2 - 1, 0.5 \cdot 1 - 0.5 \cdot (-2) - 1] = [-2.5, 0.5] \end{aligned} \tag{40}$$

   So we set $l_3 = -0.5, u_3 = 2.5$, and $l_4 = -2.5, u_4 = 0.5$.

3. **Substituting bounds** into the constraints:

$$\begin{aligned} &\hat{z}_3 \leq a_3 \cdot 2.5, \quad \hat{z}_3 \leq z_3 - (-0.5)(1 - a_3) = z_3 + 0.5(1 - a_3) \\ &\hat{z}_4 \leq a_4 \cdot 0.5, \quad \hat{z}_4 \leq z_4 - (-2.5)(1 - a_4) = z_4 + 2.5(1 - a_4) \end{aligned} \tag{41}$$

4. **The final MILP encoding:**

$$\begin{aligned} &-1 \leq x_1 \leq 1; \quad -2 \leq x_2 \leq 2 \\ &z_3 = -0.5x_1 + 0.5x_2 + 1.0, \quad z_4 = 0.5x_1 - 0.5x_2 - 1.0 \\ &\hat{z}_3 \geq z_3, \quad \hat{z}_3 \geq 0, \quad \hat{z}_4 \geq z_4, \quad \hat{z}_4 \geq 0 \\ &a_3, a_4 \in \{0, 1\} \\ &\hat{z}_3 \leq a_3 \cdot 2.5, \quad \hat{z}_3 \leq z_3 + 0.5(1 - a_3) \quad \hat{z}_4 \leq a_4 \cdot 0.5, \quad \hat{z}_4 \leq z_4 + 2.5(1 - a_4) \\ &x_5 = -\hat{z}_3 + \hat{z}_4 - 1.0, \quad x_5 \leq 0 \\ &\text{where } z_3 \in [-0.5, 2.5], z_4 \in [-2.5, 0.5], \hat{z}_3 \in [0, 2.5], \hat{z}_4 \in [0, 0.5]. \end{aligned} \tag{42}$$

1. **Solving** The MILP solver attempts to find a feasible (Sec. D.3.1) or satisfying assignment representing a counterexample violating the property. In this example, it might find $x_1 = -1, x_2 = 2$, which leads to:

$$\begin{aligned} &z_3 = -0.5(-1) + 0.5(2) + 1.0 = 2.5, \quad z_4 = 0.5(-1) - 0.5(2) - 1.0 = -2.5 \\ &a_3 = 1, \hat{z}_3 = 2.5 \quad \text{(neuron active)}, \quad a_4 = 0, \hat{z}_4 = 0 \quad \text{(neuron inactive)} \\ &x_5 = -2.5 + 0 - 1.0 = -3.5 \end{aligned} \tag{43}$$

Since $x_5 = -3.5$, this satisfies our search for $x_5 \leq 0$ and thus is a counterexample. ◊

**Problem 4.2.3.2**

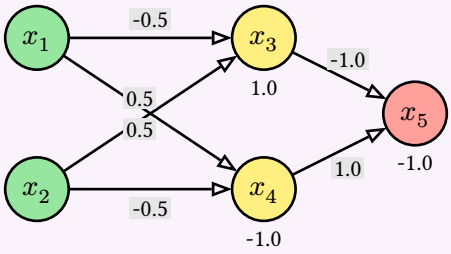



Fig. 12: A simple network (similar to Fig. 1).

Use MILP encoding and solving to check if the network Fig. 12 satisfies the property $x_5 \geq x_6$ for any inputs $x_1 \in [-1, 1], x_2 \in [-1, 1]$. Specificaly, do the following steps:

1. Write down the precondition $\varphi_{\text{in}}$ and postcondition $\varphi_{\text{out}}$.
2. Write down the MILP encoding of the network.
3. Compute the upper and lower bounds of the pre-activation values of the neurons in the hidden layer.
4. Substitute the bounds into the MILP encoding.
5. Write down the final MILP encoding of $\alpha l \wedge \varphi_{\text{in}} l \wedge \neg\{\varphi_{\text{out}}\}$.
6. Check the feasibility of the MILP encoding (e.g., using Z3) and report the result.

◊

**Problem 4.2.3.3**

As shown in Example 4.2.2.1, the upper and lower bounds of neuron $x_3$ of the network in Fig. 10 are $u_3 = 2.5$ and $l_3 = -0.5$.

1. Compute the upper and lower bounds of $x_4$ in the same example.
2. Change the weights and/or biases of the network in Fig. 10, but keep the input ranges $-1 \leq x_1 \leq 1$ and $-2 \leq x_2 \leq 2$, such that $x_3$ becomes a stable neuron (always active or always inactive). Show the new weights and/or biases, and compute the new upper and lower bounds of $x_3$.

◊

### 4.2.4 Limitations

- **Scalability**: While MILP is efficient in dealing with linear constraints, it still cannot be applied directly to real-world neural networks due to the large search space. Each ReLU application to an unstable neuron introduces a binary variable (Sec. 4.2.1), leading to exponential number of possible branches, and real-world networks can have millions of such ReLUs, making the search space intractable.
- **Limited exploitation of network structure and modern hardware**: Off-the-shelf MILP solvers are designed for arbitrary MILP problems and do not exploit specific structures of neural networks and concepts such as activation patterns (Sec. 6.1) to prune the search space. Moreover, MILP solvers, even industrial-strength ones such as Gurobi [9], are primarily CPU-based and do not leverage the massive parallelism provided by modern GPUs to improve scalability.

- **Advanced Abstraction and Heuristics**: Interval analysis is efficient and commonly used for computing neuron bounds, but cannot capture dependencies between neurons, leading to precision loss in deeper networks. SOTA NNV tools use more advanced abstract domains such as zonotopes and polytopes to improve precision (Sec. 5).

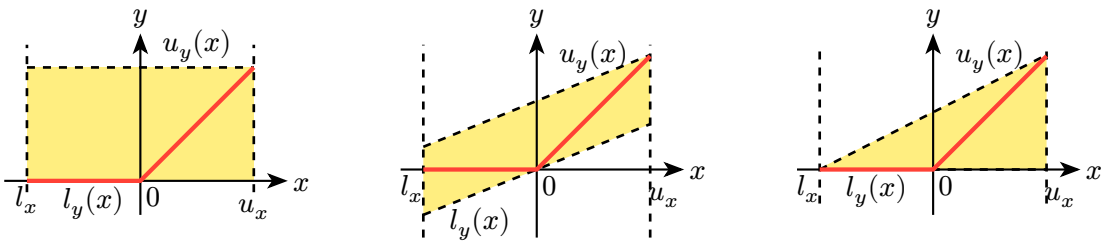
  Modern NNV tools also employ heuristics to decide which neurons to branch on and to determine stable neurons to avoid unnecessary branching. These heuristics are not available in general MILP solvers.

# 5 Abstractions

As mentioned in Sec. 4.2.1 the computation of upper and lower bounds of the neuron values help determine neuron stability and therefore can help scale NNV. Modern NNV tools explore different *abstraction* techniques to compute these bounds more precisely while balancing computational efficiency.

This chapter discusses the interval, zonotope, and polytope abstract domains commonly used in NNV. Each domain provides a different way to represent and compute the bounds of neurons, with its own trade-offs in terms of precision and computational complexity.

## 5.1 Overview and Background




(a) Interval (b) Zonotope (c) Polytope

Fig. 14: Abstractions of $\text{relu}(x) = \max(0, x)$ over $x \in [l_x, u_x]$ using (a) interval, (b) zonotope, (c) polytope abstractions.

***Abstractions for ReLU*** We use ReLU to illustrate different abstractions. Our goal is to approximate the bounds of the post-ReLU value of a neuron given the bounds of its pre-ReLU value. For example, for a ReLU neuron $y = \max(0, x)$, we want to compute the bounds $\left[l_{y(x)}, u_{y(x)}\right]$ of $y$ given the bounds $[l_x, u_x]$ of $x$.

Fig. 14 illustrates common abstractions (or *over-approximations*) for the ReLU function $y = \max(0, x)$, where $x \in [l_x, u_x]$ and $y \in \left[l_{y(x)}, u_{y(x)}\right]$. The values of ReLU are shown as points on the red line, which is non-convex and consists of two linear segments: one for $x < 0$ (where $y = 0$) and another for $x \geq 0$ (where $y = x$). To compute the bounds $l_{y(x)}$ and $u_{y(x)}$, we can use different abstractions:

1. **Interval Abstraction**: Interval represents the post-ReLU value as an interval $\left[l_{y(x)}, u_{y(x)}\right]$ where $l_{y(x)} = 0$ and $u_{y(x)} = u_x$. Interval is a simple and efficient abstraction, but it can be *imprecise*, e.g., it does not capture the relationship between the input and output of ReLU. %, and simply assumes that the output can take any value between 0 and the upper bound of the input.

   In Fig. 14(a), the yellow rectangle (box) represents the interval abstraction in the 2D space of $(x, y)$. As can be seen, this box is too large of an over-approximation (too coarse or loose), as it includes many points that are not achievable by ReLU, e.g., the point $(0.5, 0.0)$ is not on the red line.

2. **Zonotope Abstraction**: Zonotopes, commonly represented using center and generators, can capture linear relationships between variables more effectively.

In Fig. 14(b), the a parallelogram—a zonotope in 2D—is arguably tighter than the interval in Fig. 14(a). It captures the linear relationship between $x$ and $y$ and excludes points that are not achievable by ReLU, e.g., the point $(0.5, 0.0)$ that was included in the interval abstraction is not included in the zonotope. However, it still includes non-ReLU points and is also not strictly better than interval, e.g., the point $(0.5, -0.5)$ is included in the zonotope but not in the interval abstraction (or ReLU).

3. **Polytope Abstraction**: Polytopes can represent arbitrary linear constraints and provide very precise bounds. We can construct a polytope that tightly encloses the non-convex shape of the ReLU function.

   In Fig. 14(c), the polytope is shown as a trapezoid that captures the linear segments of ReLU. The lower bound is $l_{y(x)} = 0$ and the upper bound is $u_{y(x)} = u_x$.

## 5.2 Geometric Representations

Our abstract domains, e.g., interval, zonotope, and polytopes, are all geometric shapes. There are two common ways to describe a geometric shape:

- **Corner-based representation**: Define the shape by listing all of its corner points (*vertices*). For example, a rectangle in 2D is defined by its four corners, and a box in 3D is defined by its eight corners. This representation is intuitive and directly shows the boundaries of the shape.
- **Generator-based representation**: Define the shape using a *center point* and several direction vectors (*generators*) that define how the shape can stretch or tilt. Instead of listing all corners, it specifies how to reach any point in the shape by starting from the center and moving along the generators within certain limits. This representation is often more compact, especially for higher-dimensional shapes.

Fig. 15 uses corner-based and generator-based representations to represent a 2D rectangle, which is an interval abstraction over two variables. The left side shows the corner-based view, where the rectangle is defined by its 4 corners. The right side shows the generator-based view, where the rectangle is represented as a zonotope with a center point $c$ and two orthogonal generators $g_1$ and $g_2$.

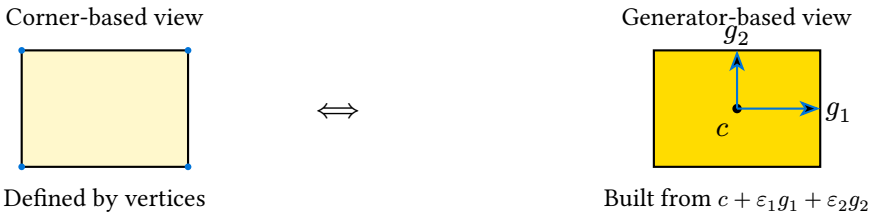



Fig. 15: Two equivalent descriptions of a 2D interval abstraction (a rectangle). Left: defined by its four corners (independent variable bounds). Right: represented as a zonotope with center $c$ and orthogonal generators $g_1$, $g_2$.

## 5.3 Interval

## 5.4 Zonotope

# 6 The Branch and Bound Search Algorithm

As shown in Sec. 4, NNV can be formulated as a satisfiability problem, solvable using a constraint solver, e.g., SMT and MILP solvers. However, such solvers do not scale to large networks due to the complexity of the underlying formulae. Thus, modern NNV techniques reframe the problem to search for *activation patterns* that satisfy the constraints, and use the *Branch-and-Bound* algorithm to explore the space of possible activation patterns.

## 6.1 Activation Pattern Search

Neural networks with piecewise linear activation functions have a special structure that can be exploited for verification. For example, each ReLU function has two *activation statuses*: active and inactive. Each status partitions the input space into two regions—one where the neuron is active and one where it is inactive. Within each region, the network behavior can be encoded as *a linear constraint*, which can be efficiently analyzed.

Thus, the NNV problem reduces to checking that none of these linear regions contains a counterexample. More specifically, we can rewrite the satisfiability formulation as a disjunction:

$$\bigvee_{p \in P} (\alpha_p \wedge \varphi_{\text{in}} \wedge \neg \varphi_{\text{out}}) \tag{44}$$

where $P$ is the set of all possible *activation patterns*—boolean assignments of activation statuses of all neurons—of the network, and $\alpha_p$ is the formula $\alpha$ restricted to the linear region defined by the activation pattern $p$. This means that $\alpha_p$ includes additional constraints that fix the activation status of each neuron according to $p$. For example, if $p$ specifies that neuron $n_i$ is active, then $\alpha_p$ includes the constraint $z_i \geq 0$, indicating that the pre-activation value $z_i$ of $n_i$ is non-negative. Similarly, if $n_i$ is inactive, then $\alpha_p$ includes $z_i < 0$.

As long as one of the disjuncts in Eq. 44 is satisfiable, i.e., a counterexample exists in one of the linear regions, the entire formula is satisfiable, indicating that the property is invalid. Conversely, if all disjuncts are unsatisfiable, the property is valid because no counterexample exists.

This allows us to break down the verification problem into smaller subproblems, each searching for an activation pattern that satisfies the formula. Modern NNV techniques all adopt this idea and search for a satisfying activation pattern to find a counterexample.

### 6.1.1 Activation Patterns

**Definition 6.1.1.1 (Activation Pattern)**

Let $N$ be a neural network with ReLU neurons $n_1, ..., n_m$. An *activation pattern* is a Boolean assignment of the activation status of a subset (or all) of the neurons. If a neuron $n_i$ is active, we set $b_i =$ `true` (meaning $z_i \geq 0$); if it is inactive, $b_i =$ `false` (meaning $z_i < 0$).

**Definition 6.1.1.2 (Complete Activation Pattern)**

A *complete activation pattern* assigns an activation status to **every** neuron in the network:

$$p = \langle b_1, b_2, ..., b_m \rangle \in \{ \texttt{true}, \texttt{false} \}^m. \tag{45}$$

♣

**Definition 6.1.1.3 (Partial Activation Pattern)**

A *partial activation pattern* assigns activation statuses to only a **subset** of the neurons:

$$q = \langle b_1, b_2, *, b_4, *, ..., b_m \rangle \in \{ \texttt{true}, \texttt{false}, * \}^m, \tag{46}$$

where $*$ means "undetermined" or "don't care". Each partial activation pattern represents multiple complete activation patterns.

♣

**Example 6.1.1.4**

Consider a network with three ReLU neurons $n_1, n_2, n_3$. A complete activation pattern might be

$$p = \langle \texttt{true}, \texttt{false}, \texttt{true} \rangle, \tag{47}$$

which means $n_1$ and $n_3$ are active while $n_2$ is inactive.

In contrast, a partial activation pattern such as

$$q = \langle \texttt{true}, \texttt{false}, * \rangle \tag{48}$$

represents **both** complete patterns $\langle \texttt{true}, \texttt{false}, \texttt{true} \rangle$ and $\langle \texttt{true}, \texttt{false}, \texttt{false} \rangle$.

◊

**Definition 6.1.1.5 (Empty Activation Pattern)**

An *empty* activation pattern is a partial pattern that assigns **no** activation statuses to any neurons:

$$p = \langle *, *, ..., * \rangle \in \{ \texttt{true}, \texttt{false}, * \}^m. \tag{49}$$

It thus represents all $2^m$ complete activation patterns.

♣

**Problem 6.1.1.6**

Consider an NN with 3 ReLU neurons:

1. How many complete activation patterns are there?
2. How many partial patterns of size 2 (i.e., fixing 2 neurons but leaving 1 unspecified) are there?
3. Give an explicit example of one partial pattern and list all the complete patterns it represents.

*Solution.*

1. $2^3 = 8$ complete patterns.
2. Choose 2 neurons to fix: $\binom{3}{2} = 3$ ways. For each, $2^2 = 4$ assignments. Total $= 12$ partial patterns.
3. Example: $p' = \{n_1 = \mathsf{true}, n_3 = \mathsf{false}\}$ represents

$\{n_1 = \mathsf{true}, n_2 = \mathsf{true}, n_3 = \mathsf{false}\}$ and $\{n_1 = \mathsf{true}, n_2 = \mathsf{false}, n_3 = \mathsf{false}\}$.

◊

**Problem 6.1.1.7**

Suppose we have a network with $m = 10$ ReLU neurons. Consider a partial pattern $p'$ that fixes 4 neurons.

1. How many complete patterns extend $p'$?
2. Suppose we restrict $p'$ further by adding 2 more neuron assignments. How many extensions now?
3. Generalize: if $p'$ fixes $k$ neurons out of $m$, how many complete patterns are there?

*Solution.*

1. $2^{10-4} = 2^6 = 64$.
2. $2^{10-6} = 2^4 = 16$.
3. General: $2^{m-k}$.

◊

### 6.1.2 Set Notation of Activation Patterns

We can use *set notation* to represent activation patterns more concisely by listing only the neurons whose activation statuses are fixed. For example, the activation pattern

$$p = \langle \texttt{true}, \texttt{false}, *, \texttt{true}, \texttt{false} \rangle \tag{50}$$

can be represented as

$$p = \{n_1 = \texttt{true}, n_2 = \texttt{false}, n_3 = *, n_4 = \texttt{true}, n_5 = \texttt{false}\}, \tag{51}$$

or more compactly

$$p = \{n_1, \overline{n_2}, n_4, \overline{n_5}\}, \tag{52}$$

where $n_i$ indicates that neuron $n_i$ is active, $\overline{n_i}$ indicates that it is inactive, and the absence of $n_3$ indicates that its activation status is unspecified.

Thus, given a set of neurons, we can construct an activation pattern by specifying which neurons are active and which are inactive. If the set includes all neurons, it is a complete pattern; otherwise, it is a partial pattern (and if it is an empty set $\emptyset$, it is the empty pattern as in Definition 6.1.1.5).

### 6.1.3 State Space Reduction

Once having an activation pattern $p$, we can simplify the formula $\alpha$ representing the network by replacing each ReLU function with a linear constraint according to the activation status specified in $p$. For example, if $p$ specifies that neuron $n_i$ is active, we replace $\mathrm{relu}(z_i)$ with $z_i \geq 0$. Otherwise, if $n_i$ is inactive, we replace $\mathrm{relu}(z_i)$ with $z_i < 0$. This gives us the formula $\alpha_p$ corresponding to the linear region defined by $p$.

A complete activation pattern $p$ defines a unique linear region of the network because with a complete pattern, $\alpha_p$ becomes a purely linear formula. In contrast, a partial activation pattern $q$ defines multiple linear regions, one for each complete pattern it represents. With $q$, $\alpha_q$ may still contain some ReLU functions that are not fixed by $q$. However, since $q$ fixes the status of some neurons, we can simplify $\alpha$ by replacing those neurons' ReLU functions with linear constraints.

In any case, given an activation pattern, we can reduce the complexity of the satisfiability check by fixing the activation status of some neurons. Thus, checking the satisfiability of $\alpha_p \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$ is easier than checking that of $\alpha \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$.

**Example 6.1.3.1**

Recall the network from Fig. 4 can be represented as the formula $\alpha$

$$\begin{aligned} \hat{x}_3 &= -0.5x_1 + 0.5x_2 + 1.0 \\ \hat{x}_4 &= 0.5x_1 - 0.5x_2 - 1.0 \\ x_3 &= \text{relu}(\hat{x}_3) \\ x_4 &= \text{relu}(\hat{x}_4) \\ x_5 &= -x_3 + x_4 - 1.0, \end{aligned} \tag{53}$$

where $\hat{x}_3$ and $\hat{x}_4$ are the pre-activation values of the ReLU neurons $x_3$ and $x_4$, respectively. Thus, if we fix the the activation status of $x_3$ and $x_4$, we can simplify $\alpha$ by replacing the ReLUs with linear constraints.

- A **complete** activation pattern specifies the status of both ReLU neurons. For instance,

  $$p = \{x_3, \overline{x_4}\} \tag{54}$$

  means $x_3$ is active while $x_4$ is inactive; that is, $\hat{x}_3 \geq 0$ (so $x_3 = \hat{x}_3$) while $\hat{x}_4 < 0$ (so $x_4 = 0$). In this case, $\alpha$ reduces to a single linear constraint:

  $$x_5 = -(-0.5x_1 + 0.5x_2 + 1.0) + 0 - 1.0 = 0.5x_1 - 0.5x_2 - 2.0. \tag{55}$$

  This reduction significantly simplifies the satisfiability check: we no longer have to reason about the non-linearity of ReLU, and can directly check whether the linear constraint is satisfiable with the input and output constraints.

- A **partial** pattern specifies only some activation statuses. For example,

  $$q = \{x_3\} \tag{56}$$

  means $\hat{x}_3 \geq 0$ but places no restriction on $\hat{x}_4$. Here, $\alpha$ reduces to:

  $$\begin{aligned} \hat{x}_4 &= 0.5x_1 - 0.5x_2 - 1.0 \\ x_4 &= \text{relu}(\hat{x}_4) \\ x_5 &= -(-0.5x_1 + 0.5x_2 + 1.0) + x_4 - 1.0. \end{aligned} \tag{57}$$

  Because $q$ does not fix the status of $x_4$, we cannot simplify the ReLU for $x_4$. Thus, the formula still contains a ReLU which splits $x_4$ into two linear regions. Note that this is still simpler than the original $\alpha$ because we have simplified the ReLU for $x_3$.

◊

**Problem 6.1.3.2**

Consider the network in Fig. 4. Suppose we fix the activation pattern $p = \{x_3, x_4\}$.

1. Provide the constraints $\alpha$ induced by $p$.
2. Consider an invalid property $x_5 > 0$ for inputs $x_1 \in [-1, 1]$, $x_2 \in [-2, 2]$. Find an activation pattern that satisfies the formula $\alpha_p \wedge \varphi_{\text{in}} \wedge \neg\varphi_{\text{out}}$. In other words, find a pattern that contains a counterexample to the property.

◊

```
1  Input: network 𝒩, property φ_in ⇒ φ_out
2  Output: unsat if property is valid, otherwise (sat, cex)

3  ActPatterns ← {∅} //initialize verification problem
4  while ActPatterns ≠ ∅
5      σ_i ← Select(ActPatterns) // process problem i^th
6      if Deduce(𝒩, φ_in ⇒ φ_out, σ_i) // check satisfiability
7          if (found a valid cex)
8              return (sat, cex)
9          v_i ← Decide(𝒩, σ_i)
           // create new activation pattern
10         ActPatterns ← ActPatterns ∪ {σ_i ∪ {v_i}; σ_i ∪ {¬v_i}}
11 return "unsat"
```

Algorithm 1: Branch and Bound (BAB) Algorithm for Neural Network Verification

## 6.2 The Branch and Bound Algorithm

Many modern NNV tools adopt the Branch-and-Bound (BAB) approach to explore the space of possible neuron activation patterns (Sec. 6.1). BAB splits (*branch*) the verification problem into smaller subproblems by fixing activation statuses of neurons, and uses abstraction (*bound*) techniques to compute upper and lower bounds on the output values for these subproblems. If the bounds, computed using abstraction (Sec. 5), indicate infeasible (cannot contain a counterexample), the subproblem is pruned from the search space. This process continues until either a counterexample is found or all subproblems are exhausted, proving the property holds.

Note that because BAB splits ReLU neurons into active and inactive cases, it is also called *"neuron-splitting"*, which contrasts with *"input-splitting"* techniques (Sec. 8.1) that partition the input space.

***Reference BaB Algorithm*** Algorithm 1 shows BAB, a reference [14] Branch and Bound architecture for NNV. BAB takes as input a ReLU-based network $\mathcal{N}$ and a formulae $\varphi_{\text{in}} \Rightarrow \varphi_{\text{out}}$ representing the property of interest. BAB maintains a set of activaion patterns (ActPatterns) that represent the current activation pattern of the network. Initially, ActPatterns is initialized with an empty activation pattern (Line 3).

In each BAB iteration $i$ (Line 4), the algorithm selects and removes an activation pattern $\sigma_i$ from ActPatternss (Line 5). It then calls Deduce to *quickly* determine if the current problem, i.e., the original satisfiability problem with activation pattern $\sigma_i$, is feasible. For example, it can use interval abstraction (Sec. 5) to quickly computes the bounds of the output values with respect to $\sigma_i$, e.g., by replacing ReLU functions according to the activation statuses specified in $\sigma_i$ (Sec. 6.1.3).

***Deduce vs. LP*** The difference between DEDUCE and LP is that the former uses abstraction to compute over-approximated bounds to determine feasibility, while the latter uses an LP solver

to check exact satisfiability. Abstraction is (very) quick but may yield false positives (declaring feasible when it is not), while LP is precise but more computationally expensive. Their combination allows BaB to efficiently prune infeasible branches while accurately checking for counterexamples.

# 7 Adversarial Attacks

# 8 Common Engineerings and Optimizations

In addition to adversarial attacks (Sec. 7), NNV tools often employ a range of optimizations and engineering techniques to improve performance. This chapter discusses some of those common techniques.

## 8.1 Input Splitting

# A Formal Methods

Neural network verification (NNV) is a rising topic that applies formal methods (FM) to ML systems, particularly neural networks (NN). This chapter explains FM techniques, how verification differs from testing and debugging, and key concepts such as specifications and common FM techniques.

## A.1 What Are Formal Methods (FM)?

FM are techniques for analyzing systems, e.g., computer software, using *mathematical models and reasoning*. In FM the behavior of a program is described and analyzed using logic and mathematics rather than informal reasoning or empirical testing. This allows FM to provide emph{guarantees} about system properties. For example, an FM might prove that a sorting algorithm always produces a correctly ordered list for any input list

***Compared to Testing*** To motivate the need for FM, consider a simple function that computes the absolute value of an integer, where the goal is to show that the function always returns a non-negative number.

```
def abs_val(x):
    if x >= 0:
        return x
    else:
        return -x
```

We might *test* `abs_val` by running it on a few inputs and checking the outputs:

```
assert(abs_val(3) = 3)
assert(abs_val(-5) = 5)
assert(abs_val(0) = 0)
...
```

The program works correctly on these inputs, returning non-negative values as expected. We could run many more tests with different inputs to increase our confidence. However, we can never test all possible integers, and therefore cannot be certain that the function works correctly for all possible inputs. This is precisely where bugs can hide, and why testing—no matter how extensive—alone cannot provide absolute guarantees about program correctness, as observed by Dijkstra:

> *"Testing can only show the presence of bugs, not their absence"*
>
> **— Edsger Dijkstra**

In contrast, an FM technique would instead reason directly about the structure of the code and atempts to prove the statement:

$$\text{For all integers } x, \text{abs\_val}(x) \geq 0. \tag{58}$$

If FM can prove this statement, we have a mathematical guarantee that the function behaves correctly for *all integers*—not just the ones we used in testing.

Thus, the *fundamental difference* between FM and testing is that FM provides mathematical guarantees about program behavior—without even running the program—, while testing can only provide empirical evidence based on a finite set of inputs that we run the program on.

## A.2 Specifications

At the heart of FM is a specification—a precise description of what it means for a system to be correct (or safe, or robust, etc.). A specification defines the properties we want to verify about a system.

Software engineers often write specifications informally in natural language, e.g., an algorithm should be "correct" and "fast", or a web server should be "robust against attacks". However, these informal specifications are often ambiguous and imprecise, making them vulnerable to misinterpretation and error. For example, what is correct? How fast is fast enough? How much perturbation should a system be able to handle to be considered "robust"?

**Problem 1.2.1 (Sorting Specification)**

Give the formal specification for a sorting function `sort(l1)` that takes a list of integers `l1` and returns a list of integers `l2`. The specification should define what it means for the output `l2` to be a correct sorting of the input `l1`.

◊

In contrast, a formal specification must be *unambiguous* and *mathematically expressible*—typically using logic. It precisely defines the conditions under which the system is considered correct.

For example, a desirable property of neural networks used for image classification is being *robust* to small perturbations of the input image. For this property, an informal specification might say:

> *"Small changes to the input image shouldn't cause the neural network to change its classification label."*

In contrast, a formal specification of robustness would instead say:

> *"For an input image $x$, for all images $x'$ such that the distance between $x$ and $x'$ is at most $\varepsilon$, the output label of a network $N$ for $x'$ must be the same as for $x$."*

This statement is more precise. It defines exactly what inputs are allowed (those within distance $\varepsilon$ of $x$) and exactly what behavior is required (the output label must remain the same). Typically, we go further and express this property in formal logic:

$$\forall x', \|x - x'\| \leq \varepsilon \implies N(x') = N(x) \tag{59}$$

Once having a formal specification like this, we can apply formal verification techniques to prove or disprove it.

### A.2.1 Pre and Post Conditions

A common way to express formal specifications is using preconditions and postconditions. A precondition describes the assumptions about the inputs to a function, while a postcondition describes the guarantees about the outputs given that the preconditions hold. Typically, preconditions and postconditions are expressed using logical formulae and together form the specification of the form:

$$P \implies Q, \tag{60}$$

which means: *"if the precondition $P$ holds for the inputs, then the postcondition $Q$ must hold for the outputs"*.

***Postcondition*** A postcondition is a logical statement that must be true after a function has executed, assuming the precondition held. It typically describes the *expected behavior* and *outcomes* of the function with respect to its inputs. For example, the postcondition for `sqrt(x)` might state that the output $y$ must be non-negative and that $y^2 = x$. For `idiv(a, b)`, the postcondition might state that the output $q$ must satisfy $a = b \cdot q + r$ for some remainder $r$ such that $0 \leq r < |b|$.

We want the postcondition to be as strong as possible, i.e., it should as precisely as possible describe the expected behavior of the function. For example, for `idiv(a, b)`, a (weak) postcondition might simply state that the output is an integer, while a (strong) postcondition would specify the exact relationship between inputs and outputs as described above. Similarly, for `sqrt(x)`, a weak postcondition might state that the output is a real number, while a strong postcondition would specify the exact relationship $y^2 = x$.

***Precondition*** A precondition is a logical statement that must be true before a function is executed. It typically lists *assumptions* and *constraints* on the inputs. For example, a `sqrt(x)` function might have the precondition that `x` must be non-negative. Similarly, an integer division function `idiv(a, b)` might have the precondition that `b` must be non-zero to avoid division-by-zero errors.

The precondition should be as weak as possible, i.e., it should not impose unnecessary restrictions or assumptions that limit the applicability of the function. Moreover, the user might not even be aware of all the assumptions needed for the function to work correctly, and therefore a strong precondition might be unrealistic. Ideally, there should be *no* precondition at all (i.e., as weak as possible), meaning the function works for all possible inputs. For example, for `sqrt(x)` we can have a weak precondition that `x` is any real number, and the postcondition would then specify that if the input is negative the output is NaN (not a number), and otherwise the output is non-negative and its square equals `x`.

**Problem 1.2.1.1 (Minimum Specification)**

Give a specification for a function `m = min(l)` that takes a list of integers `l` and returns the minimum integer `m` in the list. Clearly state the precondition and postcondition for the function.

◊

**Problem 1.2.1.2 (Duplicate)**

Give a specification for a function `l2 = remove_duplicates(l)` that takes a list of integers `l` and returns a list of integers `l2` that contains the same integers as `l` but with all duplicates removed. Clearly state the precondition and postcondition for the function.

◊

## A.3 FM Techniques

At a high level, an FM technique is an algorithm, often implemented as a software tool, that analyzes a system. The FM tool takes as *inputs*:

1. A *model* of the system to be analyzed
2. A *specification* of the desired property of the system or program

and applies a *reasoning process* to determine whether the model satisfies the specification. The tool produces an *output* indicating whether the property holds, does not hold, or is unknown.

***System Model*** The FM tool takes as input a *model* of the system to be analyzed. This model is a mathematical abstraction of the system's behavior. It may be a program written in a language such as C or Python, a neural network in ONNX format, or other representations such as state machines or transition systems. This model, e.g., the program code, is created by the programmer and is intended to capture the desired specification of the system. However, the model might contain bugs or inaccuracies that violate the intended specification, which is what the FM tool aims to detect.

**Example 1.3.1 (Clip)**

Consider the following simple system that "clips" an input value into the range $[0, 10]$. This system is modeled by the following Python function:

```python
def clip(x):
    if x < 0: return 0
    elif x > 10: return 10
    else: return x
```

◊

The formal specification of the `clip` system is:

$$\forall x \in \mathbb{R}, \quad 0 \leq \text{clip}(x) \leq 10. \tag{61}$$

**FM Reasoning Process.** Once the model and specification are given, an FM tool applies a *reasoning process*—an algorithm—to determine whether the model satisfies the specification. The next section describes several major classes of FM techniques, each with its own reasoning process.

### A.3.1 Major Classes of Formal Methods Algorithms

We now introduce the major classes of formal-methods techniques. In contrast to dynamic or testing-based analysis, which execute the program on specific inputs, these formal methods *statically* analyze the program's structure and logic to reason about all possible behaviors.

We will use the `clip` function (Example 1.3.1) as a running example to demonstrate how each technique reasons about the same model and specification.

***Model Checking*** Model checking (MC) works by *exploring all possible states* of a system and checking whether *bad states* that violate the specification are reachable. At a high level, MC constructs a state machine to model the system's behavior and systematically checks whether any execution path leads to a violation of the specification.

MC works well for systems with a finite number of states—such as hardware designs and communication protocols—because these systems have finite state spaces that can be exhaustively explored. However, MC struggles with systems involving continuous variables or infinite state spaces, e.g., a program with loops.

**Example 1.3.1.1**

For `clip`, MC would build a state machine representing all possible execution paths of the program for different inputs. It would then check whether any path leads to a state where the output is $< 0$ or $> 10$.

A standard (naïve) MC would struggle here because the number of states is huge (the input $x$ is a real number). So MC would need to discretize or *bound* the input space—e.g., by considering only integer values of $x$ from $-1000$ to $1000$—to make the reasoning more feasible.

◊

***Interactive Theorem Proving*** An interactive theorem prover (ITP) or *proof assistant* attempts to reason about the program using *logical inference rules* to construct a formal proof that the specification holds for all possible inputs. The reason it is called "interactive" is that the user is often involved to guide the proof process by providing lemmas, definitions, and strategies to help the prover construct the proof.

ITPs are powerful and can prove deep, rich properties about programs. However, they may require significant human guidance to construct the proofs and do not scale well to large or complex systems.

**Example 1.3.1.2**

For `clip`, a theorem prover might reason as follows:

1. **Case 1:** If `x < 0`, `clip` returns $0$, which is in $[0, 10]$.
2. **Case 2:** If `x > 10`, `clip` returns $10$, which is in $[0, 10]$.
3. **Case 3:** Otherwise, $0 \leq x \leq 10$, so `clip` returns `x`, which is in $[0, 10]$.

◊

***Satisfiability Checking (SAT) and Satisfiability Modulo Theories (SMT) Solving*** SAT and SMT solving are *automated reasoning* techniques[3]—also referred to as *automatic theorem proving*—that reduce the verification problem to a question of *logical satisfiability*. These solvers take as input a set of logical formulae representing the program and the *negation* of the specification, and determine whether there exists an assignment of variables that makes the formulae true (satisfiable) or not (unsatisfiable). If the formulae are unsatisfiable, the property is proved. If satisfiable, the property is disproved, and the solver provides a concrete counterexample demonstrating the violation.

SAT/SMT solving is crucial to modern formal methods and software verification tools due to its automation and ability to handle complex logical constraints. However, pure SAT/SMT solving might struggle with scalability (e.g., to analyze large neural networks). This leads to the development of hybrid techniques that combine SAT/SMT solving with other methods such as abstract interpretation (described below).

**Example 1.3.1.3**

For `clip`, a SAT/SMT solver would encode the program as logical constraints:

$$\begin{aligned}&(x < 0 \implies y = 0)\\ &\land (x > 10 \implies y = 10)\\ &\land (0 \leq x \leq 10 \implies y = x)\end{aligned} \tag{62}$$

and the negation of the specification:

$$(y < 0) \lor (y > 10) \tag{63}$$

It would then check whether these constraints are satisfiable. In this case, the solver would find that the constraints are *unsatisfiable*, proving that `clip` always returns a value in $[0, 10]$.

◊

***Abstract Interpretation*** Abstract interpretation (AbsInt) analyzes a program by automatically computing *over-approximations* of its behavior. Instead of tracking exact values, AbsInt tracks abstract properties such as ranges or signs of variables—e.g., instead of saying "`x = 5`", AbsInt might say "`x` $\in [0, 10]$" or "`x` is non-negative".

Because AbsInt is fully automated and uses over-approximations, it is efficient and scales well to large programs. However, over-approximation comes at the cost of precision: it can cause AbsInt to produce false positives by reporting potential violations that are not real bugs.

[3] *Automated reasoning* (AR) is the subfield of CS focused on algorithms that draw logical conclusions automatically, of which SAT/SMT solvers are a key example. FM is one major application of AR: it uses these reasoning engines to verify real systems against formal specifications.

**Example 1.3.1.4**

For `clip`, an AbsInt tool might reason:

1. Input `x` is in $(-\infty, +\infty)$ and output `y` is unknown initially.
2. After the `if x < 0` branch, output `y` is in $[0, 0]$.
3. After the `if x > 10` branch, output `y` is in $[10, 10]$.
4. After the else branch, output `y` is in $[0, 10]$.
5. Combining all branches, output `y` is in $[0, 10]$.

◊

For neural network verification, AbsInt is often used to compute bounds on the outputs of neural network layers given bounds on the inputs (Sec. 5). AbsInt is necessary because neural networks are very large and have complex nonlinear functions such as ReLU that are difficult to analyze exactly.

### A.3.2 Soundness and Completeness in FM

Two core concepts in FM are *soundness* and *completeness*. These describe the correctness and power of an FM algorithm.[4]

**Definition 1.3.2.1**

An FM algorithm is **sound** if every property it claims to prove is actually true. In other words, if it says *"this property holds in this system,"* then the property truly holds. Conversely, an FM algorithm is *not* sound if it can claim that a false property is true.

♣

**Definition 1.3.2.2**

An FM algorithm is **complete** if it can prove every true property. That is, if the property holds, the verifier will always be able to prove it. Conversely, an FM algorithm is **incomplete** if there exist true properties that it cannot prove.

♣

Some notes:

- While a sound FM algorithm never claims a false property is true, it *may* claim that a true property is false (i.e., it can give false counterexamples). Thus, an extremely conservative FM algorithm that claims every property is false is still sound. Similarly, an FM algorithm that returns "unknown" for every property is also sound.

  Similarly, while a complete FM algorithm can prove every true property, it may also claim that a false property is true (i.e., it can produce false proofs). Thus, an extremely reckless (and untrustworthy) FM algorithm that claims every property is true is still complete. An FM algorithm that returns "unknown" for every property is also complete.

[4] Mike Hicks wrote a good blog post on this topic: http://www.pl-enthusiast.net/2017/10/23/what-is-soundness-in-static-analysis/.

- It is important to distinguish between an FM algorithm and its implementation (the software tool). An FM algorithm may be sound (or complete), but its implementation—typically written by humans (or AI nowadays)—may contain bugs that cause it to produce unsound (or incomplete) results.

Ideally, an FM algorithm should be both sound and complete: never proving a false property and able to prove every true property. However, achieving both is often computationally infeasible for complex systems. Between the two, soundness is typically prioritized in safety-critical settings because it is better to be unsure about a system (and say "unknown", or even claim that it is unsafe) than to incorrectly claim that it is safe when it is not. Thus, most practical FM algorithms are designed to be *sound but incomplete*.

# B Logics

The topic of neural network verification (NNV) relies heavily on two fundamental areas: logic (to formalize and reason about properties as logical formulae) and optimization (to formulate and reason about numerical constraints). This chapter provides the necessary background on both topics in detail, starting with propositional logic and satisfiability, and then moving to linear and mixed-integer programming.

## B.1 Propositional Logic and Satisfiability

Propositional, or Boolean, logic is the branch of logic that studies formulae built from Boolean variables, where each variable can take only the values `True` or `False`. These formulae are combined using logical connectives such as `and`, `or`, and `not`, and their evaluation always reduces to a single truth value.

### B.1.1 Syntax

A *propositional variable* is a symbol (e.g., $x, y, z$). Logical formulae are built *inductively* from these variables using logical connectives as follows:

- **Variables:** Any propositional variable $p$ is a formula.
- **Constants:** $\top$ and $\bot$ are formulae.
- **Negation:** If $\varphi$ is a formula, then so is $\neg\varphi$.
- **Binary connectives:** If $\varphi, \psi$ are formulae, then so are $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, and $(\varphi \Rightarrow \psi)$.

**Example 2.1.1.1**

$$(x \vee y) \wedge (\neg x \vee z) \tag{64}$$

is a formula involving three variables $x, y, z$. ◊

**Special connectives.** In addition to the standard connectives, we define some special connectives that are often useful:

- *Implication*: $\varphi \to \psi \equiv \neg\varphi \vee \psi$
- *Biconditional* (or *equivalence*): $\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi)$

Notice here that we purely define the *syntax* of propositional logic, which allows us to construct valid formulae. We will discuss the *semantics* or meanings of formulae in Sec. B.1.2.

**Conjunctive Normal Form (CNF).** A formula is in CNF if it is a *conjunction* of *clauses*, where each clause is a *disjunction* of *literals*, where a literal is either a variable $p$ or its negation $\neg p$.

**Example 2.1.1.2**

The formula

$$(x \vee y) \wedge (\neg x \vee z) \tag{65}$$

is in CNF. Each clause $(x \vee y)$, $(\neg x \vee z)$ is a disjunction of literals.

◊

**Transforming to CNF.** Any formula can be transformed into CNF. The transformation process generally involves the following steps:

1. Eliminate biconditionals and implications.
2. Move negations inward using De Morgan's laws.
3. Distribute disjunctions over conjunctions.

**Problem 2.1.1.3**

Transform the following formula into CNF:

$$(x \to y) \wedge (y \to z) \tag{66}$$

◊

### B.1.2 Semantics

A logical formula in propositional logic has a truth value, which can be either `True` or `False`. For example, the formula $x \vee \neg x$ is always `True`, while the formula $x \wedge y$ is `True` if both $x$ and $y$ are `True`, and `False` otherwise. This is the *semantics* of the formula.

An *assignment* $\sigma$, which is a mapping from propositional variables to truth values, evaluates each formula to `True` or `False`. The truth value of a propositional formula over the connectives $\top, \bot, \neg, \wedge, \vee, \Rightarrow$ under an assignment $\sigma$ is defined recursively:

$$\begin{aligned}
\sigma(\top) &= \text{True}, \\
\sigma(\bot) &= \text{False}, \\
\sigma(\neg\varphi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{False}, \\
\sigma(\varphi \wedge \psi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{True and } \sigma(\psi) = \text{True}, \\
\sigma(\varphi \vee \psi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{True or } \sigma(\psi) = \text{True}, \\
\sigma(\varphi \Rightarrow \psi) = \text{True} \quad &\text{iff} \quad \sigma(\varphi) = \text{False or } \sigma(\psi) = \text{True}.
\end{aligned} \tag{67}$$

**Problem 2.1.2.1**

For the formula

$$(x \vee y) \wedge (\neg x \vee z), \tag{68}$$

evaluate its truth value under all $2^3 = 8$ possible assignments of $x, y, z$.

◊

## B.2 Satisfiability and Validity

**Definition 2.2.1 (Satisfiability)**

A formula $\varphi$

- is *satisfiable* if there exists **some** assignment $\sigma$ that satisfies it, i.e., $\exists \sigma.\ \sigma(\varphi) = \text{True}$.
- is *unsatisfiable* if **no** assignment satisfies it, i.e., $\forall \sigma.\ \sigma(\varphi) = \text{False}$.
- is *valid* if **all** assignments satisfy it, i.e., $\forall \sigma.\ \sigma(\varphi) = \text{True}$.

♣

**Example 2.2.2**

The formula $p$ is satisfiable (i.e., $\exists \sigma.\ \sigma(p) = \text{True}$).

The formula

$$(p) \wedge (\neg p) \tag{69}$$

is unsatisfiable (no assignment works). In contrast,

$$(p \vee \neg p) \tag{70}$$

is valid.

◊

**Problem 2.2.3**

Show that $(\varphi \Rightarrow \psi)$ is equivalent to $(\neg\varphi \vee \psi)$. (Hint: construct a truth table.)

◊

**Problem 2.2.4**

Assume $p, q$ are boolean variables.

1. Can a formula be satisfiable but not valid? If yes, provide an example of such a formula over $p, q$.
2. Can a formula be valid but not satisfiable? If yes, provide an example of such a formula over $p, q$.
3. Give a formula over $p, q$ that is satisfiable under exactly 1 assignment.
4. Let $\varphi$ be a formula over $p, q$. If $\varphi$ is valid, how many assignments satisfy it?

◊

### B.2.1 SAT Checking

A *SAT solver* takes a propositional formula, typically in CNF, as input and determines its satisfiability by searching for a satisfying assignment $\sigma$ that makes the formula true. If one exists it returns SAT (and optionally $\sigma$); otherwise it reports unsat.

**Example 2.2.1.1**

$$\begin{aligned} \varphi_1 &= (x \vee y) \wedge (\neg x \vee z) \quad \Rightarrow \text{sat} \\ \varphi_2 &= (x) \wedge (\neg x) \quad \Rightarrow \text{unsat} \end{aligned} \tag{71}$$

◊

**Problem 2.2.1.2**

Use the Z3 solver to check whether

$$(x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \tag{72}$$

is satisfiable. Provide a satisfying assignment if it exists.

◊

### B.2.2 Validity Checking

A formula $\alpha$ is valid if all assignments satisfy it. This is equivalent to its negation $\neg\alpha$ being unsatisfiable:

$$\text{valid}(\alpha) \iff \text{unsat}(\neg\alpha). \tag{73}$$

Thus, to check that $\alpha$ is valid, we can use a SAT solver to check that $\neg\alpha$ is UNSAT.

### B.2.3 Complexity of SAT

SAT checking (and therefore validity checking) of propositional formulae is NP-Complete. This means it is unlikely that an efficient (polynomial-time) algorithm exists for all SAT instances, and in the worst case any SAT solving algorithm may need to explore an exponential number of assignments. SAT solvers therefore employ heuristics and optimizations to handle practical instances efficiently, even though worst-case complexity remains exponential. We discuss DPLL, a popular SAT solving algorithm, next.

## B.3 Satisfiability Modulo Theories (SMT)

SAT solvers only reason about propositional formulae, which contain only Boolean variables and logical operators such as $\wedge, \vee$. However, in verification problems we often need to reason about arithmetic constraints such as $2x + 3y \leq 7$, $x \geq 0$, $x, y \in \mathbb{Z}$.

This leads to *Satisfiability Modulo Theories (SMT)*:

- SMT generalizes propositional logic by adding background theories (e.g., linear arithmetic, bit-vectors, arrays).
- An SMT formula mixes Boolean structure with constraints from these theories.

**Example 2.3.1**

Is the formula $2x = 1$ satisfiable?

The answer depends on the chosen theory. Over the reals ($\mathbb{R}$) it is SAT (e.g., $x = 0.5$), but over the integers ($\mathbb{Z}$) it is UNSAT.

◊

## B.4 SMT Solvers

Popular SMT solvers include Z3 (Microsoft Research), CVC5, and dReal. They combine:

- A SAT solver for the Boolean skeleton.
- A *theory solver* or T-solver (e.g., linear arithmetic) for the numeric parts.

The SAT solver guesses a truth assignment for the Boolean structure. The T-solver checks whether the corresponding arithmetic constraints are feasible. This loop continues until either a satisfying assignment is found or UNSAT is proven.

**Problem 2.4.1**

Decide the satisfiability of:

$$(x > 3) \land (y < 2) \land (x + y < 1). \tag{74}$$

First reason informally by hand, then confirm with an SMT solver such as Z3.

◊

### B.4.1 Z3 SMT Solver

Z3 is a well-known SMT solver developed by Microsoft Research. Here we show how to use it to check propositional formulae and SMT formulae involving linear arithmetic constraints.

**Installing.** You can install Z3 using your package manager:

```
brew install z3          # Homebrew
pip install z3-solver    # pip
apt install z3 python3-z3  # apt (Debian-based Linux)
conda install z3solver   # conda
```

Then try its Python interface:

```
from z3 import *
x = Int('x')
y = Int('y')
solve(x > 2, y < 10, x + 2*y = 7)
```

### Example 2.4.1.1 (Propositional Logic)

We use Z3 to check satisfiability of propositional formulae and generate counterexamples.

```
from z3 import *

x, y = Bools('x y')
formula = And(Or(x, y), Or(Not(x), y))
s = Solver()
s.add(formula)
print(s.check())   # output: sat
print(s.model())   # a possible model is {x=True, y=True}
```

◊

### Problem 2.4.1.2

Modify the code in the propositional logic example above to check

$$(x \wedge \neg x). \tag{75}$$

Confirm that the result is `unsat`.

◊

### Example 2.4.1.3 (Linear Constraints)

We now use Z3 as an SMT solver to check linear constraints.

```
from z3 import *
x, y = Reals('x y')

# example 1
s = Solver()
s.add(x > 0, y > 0, 2*x + y <= 1)
print(s.check())  # Output: unsat

# example 2
s = Solver()
s.add(x + y <= 4, x >= 0, y >= 0)
print(s.check())  # Output: sat
print(s.model())  # Output: {x=0, y=0}

# example 3
x, y = Ints('x y')
s = Solver()
s.add(Implies(x > 3, y > 2))
print(s.check())  # Output: sat
print(s.model())  # Output: {x=0, y=0}
```

◊

**Problem 2.4.1.4**

Use Z3 to check whether the constraints

$$x + y = 5, \quad x \geq 0, \quad y \geq 0 \tag{76}$$

are satisfiable, and if so, ask Z3 for a model.

◊

**Problem 2.4.1.5**

Use Z3 to check the formula given in the SMT example above. Do it for both the reals and integers.

◊

**Problem 2.4.1.6**

Use Z3 to formulate and check the statement *"If* $x > 3$ *then* $y > 2$ *AND* $x \leq 1$*"*. Is it satisfiable? What model does Z3 return?

◊

**Example 2.4.1.7 (DNN-style checking)**

Consider a one-neuron ReLU: $y = \max(0, x - 1)$. We want to check if the property *"For all* $x \leq 0$*, we have* $y = 0$*"* holds.

```
x, y = Reals('x y')
s = Solver()

# y = max(0, x-1)
s.add(y >= x-1, y >= 0)
s.add(Or(y = x-1, y = 0))  # ReLU
s.add(x <= 0, y != 0)        # Negation of property

print(s.check())  # Output: unsat, property holds
```

◊

**Problem 2.4.1.8**

Modify the above to check the property: *"For all* $x \geq 2$*, we have* $y \geq 1$*."* What does Z3 return?

◊

# C SAT Solving Algorithms

## C.1 DPLL

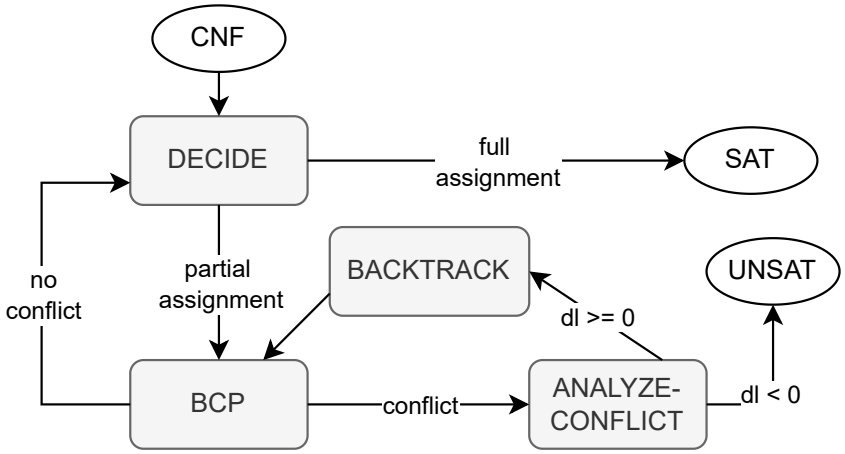



Fig. 16: The DPLL algorithm for SAT solving.

Fig. 16 gives an overview of the **DPLL** algorithm—a SAT solving technique introduced in 1961 by Davis, Putnam, Logemann, and Loveland. DPLL is an iterative algorithm that takes as input a propositional formula and (i) decides an unassigned variable and assigns it a truth value, (ii) performs Boolean constraint propagation (BCP, also called Unit Propagation), which detects single-literal clauses that either force a literal to be true or give rise to a conflict, (iii) analyzes the conflict to backtrack to a previous decision level `dl`, and (iv) erases assignments at levels greater than `dl` to try new assignments.

These steps repeat until DPLL finds a satisfying assignment and returns `sat`, or decides that it cannot backtrack (`dl = -1`) and returns `unsat`.

### C.1.1 Decide

The *Decide* step selects an unassigned variable and assigns it a truth value. Decision is the main source of state-space explosion in DPLL because it uses random choices or heuristics (e.g., VSIDS) to select the variable and truth value. Thus, value assignments can be *wrong*, leading to conflicts later on.

In addition, each decision creates a new *decision level*. The decision level starts at 0 and increases by 1 with each new assignment. Decision level is used to manage backtracking and erasing of assignments when conflicts occur.

**Example 3.1.1.1 (Simple Decision)**

Consider the CNF:

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3). \tag{77}$$

All variables are initially unassigned. DPLL may choose:

$$\text{Decide: } \; x_1 = \text{True} \quad (\text{Decision level } \; 1). \tag{78}$$

◊

**Example 3.1.1.2 (Decision Using a Heuristic)**

Given:

$$\varphi = (x_1 \vee x_3) \wedge (x_3 \vee x_4) \wedge (\neg x_3 \vee x_2), \tag{79}$$

a heuristic such as VSIDS may choose the most frequent variable:

$$\text{Decide:} \quad x_3 = \text{False}. \tag{80}$$

◊

### C.1.2 Boolean Constraint Propagation (BCP)

BCP detects *unit clauses*, i.e., clauses with exactly one unassigned literal and all other literals set to False. Such clauses force the remaining literal to be assigned True.

BCP is very useful because it can determine variable assignments directly without guessing (which is Decide as described in Sec. C.1.1). For example, if we have the clause $(\neg x_1 \vee x_2)$ and $x_1$ is True, then BCP forces $x_2$ to be True to satisfy the clause.

BCP is often invoked after each decision to propagate the consequences of the assignment.

**Example 3.1.2.1 (Single Propagation)**

$$\varphi = (x_1) \wedge (\neg x_1 \vee x_2). \tag{81}$$

The unit clause $(x_1)$ forces $x_1 = \text{True}$. Now $(\neg x_1 \vee x_2)$ becomes $(\text{False} \vee x_2)$, hence $x_2 = \text{True}$.

◊

**Example 3.1.2.2 (Propagation Leading to Conflict)**

$$\varphi = (x_1) \wedge (\neg x_1). \tag{82}$$

BCP assigns $x_1 = \text{True}$, immediately contradicting $(\neg x_1)$.

◊

**Example 3.1.2.3 (Cascade of Propagations)**

$$\varphi = (x_1) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3). \tag{83}$$

BCP yields:

$$x_1 = \text{True} \Rightarrow x_2 = \text{True} \Rightarrow x_3 = \text{True}. \tag{84}$$

◊

### C.1.3 Conflict Analysis and Backtracking

A *conflict* occurs when the formula is False under the current assignment $\sigma$. Since the formula is in CNF, a conflict arises when at least one clause is False (i.e., all its literals are False).

**Example 3.1.3.1**

$$\varphi = (\neg x_1 \vee x_2) \wedge (\neg x_2) \wedge (x_1). \tag{85}$$

Assignments $x_1 = \text{True}$ and $x_2 = \text{False}$ make $(\neg x_1 \vee x_2) = (\text{False} \vee \text{False})$, producing a conflict.

◊

**Conflict Analysis.** Conflict analysis explains why the conflict occurred and generates a *learned clause* that prevents the solverfrom revisiting the same conflicting assignment. Most modern solvers use the 1st-UIP (Unique Implication Point) learning scheme.

**Example 3.1.3.2 (Simple Learned Clause)**

$$(x_1) \wedge (\neg x_1). \tag{86}$$

The conflict directly yields the learned clause $\neg x_1 \vee x_1$. Because this is always false, the solver concludes the formula is unsatisfiable.

◊

**Example 3.1.3.3 (UIP Conflict Analysis)**

Assume decision levels:

$$x_1 = \text{True} \quad (dl = 1), \quad x_2 = \text{False} \quad (dl = 2). \tag{87}$$

Clauses:

$$(\neg x_1 \vee x_2 \vee x_3), \quad (\neg x_2 \vee x_3), \quad (\neg x_3). \tag{88}$$

Propagation yields $x_3 = \text{False}$, causing a conflict with $(\neg x_3)$. The learned clause (1st-UIP) is:

$$(\neg x_2 \vee \neg x_1). \tag{89}$$

◊

**Backtracking.** Given a learned clause, DPLL backtracks to the decision level at which the clause becomes unit.

**Example 3.1.3.4 (Non-Chronological Backtracking)**

Decision levels:

$$dl = 1 : x_1 = \text{True}, \quad dl = 2 : x_2 = \text{True}, \quad dl = 3 : x_3 = \text{False}. \tag{90}$$

Conflict analysis produces the learned clause $(\neg x_1 \vee x_3)$. The highest decision level in the clause except the most recent UIP is 1, so the solver backtracks to level 1, undoing assignments to $x_2$ and $x_3$.

◊

## C.2 CDCL

Modern DPLL solving improves the original version with Conflict-Driven Clause Learning (CDCL) [15], [16], [17]. DPLL with CDCL *learns new clauses* to avoid past conflicts and backtrack more intelligently (e.g., using non-chronological backjumping). Due to its ability to learn new clauses, CDCL can significantly reduce the search space and allow SAT solvers to scale to large problems.

In the following, whenever we refer to DPLL hereafter, we mean DPLL with CDCL.

## C.3 DPLL(T)

DPLL(T) [18] extends DPLL for propositional formulae to check SMT formulae involving non-Boolean variables, e.g., real numbers and data structures such as strings, arrays, and lists. DPLL(T) combines DPLL with dedicated *theory solvers* to analyze formulae in those theories.[5] For example, to check a formula involving linear arithmetic over the reals (LRA), DPLL(T) may use a theory solver that applies linear programming to check the constraints in the formula.

Modern DPLL(T)-based SMT solvers such as Z3 [8] and CVC4 [19] include solvers supporting a wide range of theories including linear arithmetic, nonlinear arithmetic, strings, and arrays [20].

[5] SMT stands for Satisfiability Modulo Theories; the T in DPLL(T) stands for Theories.

# D Linear Programming (LP)

Linear Programming (LP) and its generalization Mixed-Integer Linear Programming (MILP) form the optimization backbone of many NNV techniques. This chapter provides an overview of LP and MILP in the context of NNV.

## D.1 Linear Constraints and Objectives

At a high level, LP is a method for optimizing an objective with respect to certain constraints. In LP, both the objective and the constraints are *linear*.

**Linear Constraints.** Linear constraints are inequalities involving linear combinations of variables. A linear inequality has the general form

$$c_1x_1 + c_2x_2 + ... + c_nx_n \leq b, \tag{91}$$

where $c_i, b \in \mathbb{R}$. These constraints limit the feasible values the variables $x_i$ can take. The set of points satisfying all constraints is called the *feasible region*.

**Example 4.1.1**

Consider the following linear constraints:

$$x + y \leq 4, \quad x \geq 0, \quad y \geq 0. \tag{92}$$

The feasible region is a triangle with vertices at $(0, 0)$, $(4, 0)$, $(0, 4)$.

◊

**Linear Objective Functions.** A linear objective function $z$ has the general form

$$z(x_1, x_2, ..., x_n) = c_1x_1 + c_2x_2 + ... + c_nx_n, \tag{93}$$

where $c_i \in \mathbb{R}$ are coefficients and $x_i \in \mathbb{R}$ are decision variables.

**Optimizing Goals.** The goal is to optimize (maximize or minimize) $z$ subject to a set of linear constraints:

$$\begin{aligned} &\text{maximize} \quad z \\ &\text{subject to} \\ &\quad \{c_1x_1 + ... + c_nx_n \leq b, ...\} \end{aligned} \tag{94}$$

**Example 4.1.2**

Maximize $1.5x + 2y$ subject to $x + y \leq 4$, $x \geq 0$, $y \geq 0$.

Solving for the intersection points gives the vertices of the feasible region: $(0, 0)$, $(4, 0)$, $(0, 4)$.

Evaluating the objective at each vertex:

| $x$ | $y$ | $z = 1.5x + 2y$ |
|---|---|---|
| 0 | 0 | 0 |
| 4 | 0 | 6 |
| 0 | 4 | 8 |

The maximum value of $z$ is 8, achieved at vertex $(0, 4)$.

◊

**Example 4.1.3**

Minimize $z = x + y$ subject to $2x + y \geq 3$, $x + 2y \geq 4$, $x \geq 0$, $y \geq 0$.

Solving the boundary equations gives the vertices of the feasible region: $(0, 3)$, $(4, 0)$, $\left(\frac{2}{3}, \frac{5}{3}\right)$.

Evaluating the objective at each vertex:

| $x$ | $y$ | $z = x + y$ |
|---|---|---|
| 0 | 3 | 3 |
| 4 | 0 | 4 |
| $\frac{2}{3}$ | $\frac{5}{3}$ | $\frac{7}{3}$ |

The minimum value of $z$ is $\frac{7}{3}$, achieved at $\left(\frac{2}{3}, \frac{5}{3}\right)$.

◊

**Problem 4.1.4**

Maximize $z = 4x + 5y$ subject to $x + y \leq 20$, $2x + 4y \leq 72$.

1. Identify the corner points.
2. Evaluate the objective function at each corner point.

◊

## D.2 Mixed-Integer Linear Programming (MILP)

LP assumes continuous variables over real numbers. A mixed-integer linear program—MILP—extends LP by requiring some variables to be integers (often binary 0 or 1). This is useful for modeling discrete decisions and logical constraints, e.g., on/off decisions, yes/no choices, and active/inactive ReLU.

**Linear Constraints.** In MILP, a linear constraint can involve both integer and continuous decision variables:

$$c_1x_1 + ... + c_nx_n + d_1y_1 + ... + d_my_m \leq b, \tag{95}$$

where $x_i \in \mathbb{R}$ are continuous variables, $y_j \in \mathbb{Z}$ are integer variables, and $c_i, d_j, b \in \mathbb{R}$.

**Objective Function.** A linear objective function in MILP:

$$z = c_1x_1 + ... + c_nx_n + d_1y_1 + ... + d_my_m. \tag{96}$$

**Optimizing Goals.**

$$\begin{aligned} &\text{maximize} \quad z(x_1, ..., x_n, y_1, ..., y_m) \\ &\text{subject to} \\ &\quad \{c_1x_1 + ... + c_nx_n + d_1y_1 + ... + d_my_m \leq b\} \end{aligned} \tag{97}$$

**Example 4.2.1**

A company sells a chair for \$50 and a table for \$120. Production costs are \$20 per chair and \$70 per table. It takes 3 hours and 5 units of wood to produce a chair, and 1 hour and 20 units of wood to produce a table.

Maximize profit without exceeding a monthly budget of 200 units of wood, 80 hours of labor, and \$800 in production costs.

**Decision variables:** $x$ = number of chairs, $y$ = number of tables.

**Objective:**

$$P = 50x + 120y - 20x - 70y = 30x + 50y. \tag{98}$$

**Constraints:**

$$\begin{aligned} 5x + 20y \leq 200 \quad &\Rightarrow \quad x + 4y \leq 40, \\ 3x + y \leq 80, \quad & \\ 20x + 70y \leq 800 \quad &\Rightarrow \quad 2x + 7y \leq 80, \\ x \geq 0, \quad y \geq 0. \quad & \end{aligned} \tag{99}$$

The feasible vertices are $(0, 0)$, $(0, 10)$, $\left(\frac{80}{3}, 0\right)$, and $\left(\frac{280}{11}, \frac{40}{11}\right)$. Evaluating $P$:

| $x$ | $y$ | $P = 30x + 50y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 10 | 500 |
| $\frac{80}{3}$ | 0 | 800 |
| $\frac{280}{11}$ | $\frac{40}{11}$ | 945.45 |

The maximum is \$945.45 at $\left(\frac{280}{11}, \frac{40}{11}\right)$. Since fractional production is not possible, the company should produce 24 chairs and 4 tables per week for a profit of \$920.00.

### D.2.1 Encoding Binary Variables

MILP problems often involve conditions where the answer *depends* on some condition. We can encode such logical implications using *binary variables* ($z \in \{0, 1\}$) and a *large constant* $M$ (e.g., $\infty$).

**Example 4.2.1.1**

To encode "if $z = 1$ then $x \geq 5$", use:

$$x \geq 5 - M(1 - z), \quad z \in \{0, 1\}. \tag{100}$$

This works because:

- $z = 1 \Rightarrow x \geq 5 - M(0) \Rightarrow x \geq 5$.
- $z = 0 \Rightarrow x \geq 5 - M$, which is vacuously true for large $M$.

◊

**Problem 4.2.1.2**

Encode the disjunction $x \geq 5 \vee x \leq 3$ in MILP.

◊

*Solution.* Use two constraints:

- $x \geq 5 - M(1 - z)$ (if $z = 1$, this enforces $x \geq 5$)
- $x \leq 3 + Mz$ (if $z = 0$, this enforces $x \leq 3$)

**Example 4.2.1.3 (Encoding ReLU in MILP)**

To encode the ReLU activation function $y = \max(0, x)$, use:

$$\begin{aligned} &y \geq x, \\ &y \geq 0, \\ &y \leq x + M(1 - z), \\ &y \leq Mz, \\ &z \in \{0, 1\}. \end{aligned} \tag{101}$$

This works because $z = 1 \Rightarrow y = x$ and $z = 0 \Rightarrow y = 0$.

◊

**Problem 4.2.1.4**

Consider again the ReLU encoding above but now $x$ has bounds $L \leq x \leq U$. Modify the MILP encoding so that $L$ and $U$ are used *directly* instead of a large constant $M$.

◊

*Solution.*

$$
\begin{aligned}
&y \geq x, \\
&y \geq 0, \\
&y \leq x + (U - x)(1 - z), \\
&y \leq Uz, \quad z \in \{0, 1\}, \\
&L \leq x \leq U.
\end{aligned} \tag{102}
$$

## D.3 Using Z3 to Solve LP and MILP

We can use Z3's optimization capabilities to solve both LP and MILP problems. In practice, dedicated solvers such as Gurobi or CPLEX are preferred for large problems, but Z3 is effective for demonstration purposes.

**Example 4.3.1**

We solve the LP example from Example 4.1.2 using Z3:

```
from z3 import *

x, y = Reals("x y")
opt = Optimize()
opt.add(x + y <= 4, x >= 0, y >= 0)

z = 1.5*x + 2*y
h = opt.maximize(z)

if opt.check() = sat:
    print("Optimal sol:", opt.model())  # [x = 0, y = 4]
    print("Max value:", opt.upper(h))   # 8
```

◊

**Problem 4.3.2**

Solve Problem 4.1.4 using Z3.

◊

**Problem 4.3.3**

Solve Example 4.2.1 using Z3.

◊

**Problem 4.3.4**

Find two interesting MILP problems online, formulate each as a MILP, solve by hand (showing all steps), and implement in Z3.

For each problem: write down the problem statement and cite the source, clearly state the objective and constraints, and document the Z3 code with comments.

◊

**Problem 4.3.5**

Minimize $z = x + y$ subject to:

$$x + 2y \geq 3, \quad 3x + y \geq 3, \quad x, y \geq 0, \quad x, y \in \mathbb{R}. \tag{103}$$

Remember to:

1. State the constraints and objective function.
2. Find the corner or candidate points.
3. Evaluate the objective at each corner point.
4. State the optimal solution.

◊

### D.3.1 Using LP as Feasibility Checking

In addition to optimization, LP can be used for *feasibility checking* of linear constraints: are there any values for the variables that satisfy all the constraints? This is achieved by running the LP solver with a constant objective (e.g., "minimize 0"), in which case it tries to find *any* point in the feasible region or show that *none* exists. This makes LP solving useful for property checking and verification.

The main difference between LP feasibility and SMT satisfiability checking is the type of constraints they handle. SMT supports richer logics involving booleans, integers, nonlinear arithmetic, and other theories, while LP feasibility is limited to linear equalities/inequalities over real numbers. For NNV problems that involve only linear real arithmetic, LP feasibility is sufficient and often more efficient.

**Example 4.3.1.1**

For the constraints $\{x + y \leq 4, x \geq 0, y \geq 0\}$, running an LP solver with objective $\min 0$ will return a point (e.g., $x = 0, y = 0$) within the feasible region.

◊

# E Programming Assignments

## E.1 PA1

# Bibliography


[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[2] ONNX Community, "ONNX: Open Neural Network Exchange." 2017.

[3] S. Demarchi *et al.*, "Supporting Standardization of Neural Networks Verification with VNNLIB and CoCoNet.," in *FoMLAS@ CAV*, 2023, pp. 47–58.

[4] A. Tacchella, L. Pulina, D. Guidotti, and S. Demarchi, "The international benchmarks standard for the Verification of Neural Networks." [Online]. Available: https://www.vnnlib.org/

[5] C. Brix, S. Bak, T. T. Johnson, and H. Wu, "The Fifth International Verification of Neural Networks Competition (VNN-COMP 2024): Summary and Results." 2024. doi: 10.48550/arXiv.2412.19985.

[6] C. Barrett, A. Stump, C. Tinelli, and others, "The smt-lib standard: Version 2.0," in *Proceedings of the 8th international workshop on satisfiability modulo theories (Edinburgh, England)*, 2010, p. 14.

[7] X. Huang, M. Kwiatkowska, S. Wang, and M. Wu, "Safety verification of deep neural networks," in *International conference on computer aided verification*, 2017, pp. 3–29. doi: 10.1007/978-3-319-63387-9_1.

[8] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340. doi: 10.1007/978-3-540-78800-3_24.

[9] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual." [Online]. Available: https://www.gurobi.com/

[10] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *International Conference on Computer Aided Verification*, 2017, pp. 97–117. doi: 10.1007/978-3-319-63387-9_5.

[11] M. Sälzer and M. Lange, "Reachability in simple neural networks," *Fundamenta Informaticae*, vol. 189, 2023.

[12] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–39, 2018.

[13] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[14] E. Y. Nakagawa, M. Guessi, J. C. Maldonado, D. Feitosa, and F. Oquendo, "Consolidating a process for the design, representation, and evaluation of reference architectures," in *2014 IEEE/IFIP Conference on Software Architecture*, 2014, pp. 143–152.

[15] R. J. Bayardo Jr and R. Schrag, "Using CSP look-back techniques to solve real-world SAT instances," in *Aaai/iaai*, 1997, pp. 203–208.

[16] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A search algorithm for propositional satisfiability," *IEEE Transactions on Computers*, vol. 48, no. 5, pp. 506–521, 1999.

[17] J. Marques Silva and K. Sakallah, "GRASP-A new search algorithm for satisfiability," in *Proceedings of International Conference on Computer Aided Design*, 1996, pp. 220–227. doi: 10.1109/ICCAD.1996.569607.

[18] R. Nieuwenhuis, A. Oliveras, and C. Tinelli, "Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)," *Journal of the ACM (JACM)*, vol. 53, no. 6, pp. 937–977, 2006, doi: 10.1145/1217856.1217859.

[19] C. Barrett *et al.*, "Cvc4," in *International Conference on Computer Aided Verification*, 2011, pp. 171–177. [Online]. Available: https://dl.acm.org/doi/10.5555/2032305.2032319

[20] D. Kroening and O. Strichman, *Decision procedures*. Springer, 2008. [Online]. Available: https://dl.acm.org/doi/10.5555/1391237